

Temporal weakening of multidimensional trait–growth relationships among tree seedlings

28 September 2026

Contents

1. Data loading
2. PCA trait axes (Figure 2)
 - Figure 2. Above- and belowground trait axes
3. Year-specific combination-model framework
 - 3.1 Community-level models
 - 3.2 Species-level models
4. H1: Growth predictability
 - Figure 3. Temporal change in growth predictability
5. H2-H3: Relative contributions
 - Figure 4. Temporal change in relative contributions
6. Robustness analysis
7. Supplementary effect sizes
 - Figure 5. Temporal change in main effects
 - Figure 6. Temporal change in interaction effects
- Figure S8. Year-specific global-model robustness
- Reproducibility check

1. Data loading

The public data file contains one row per seedling ($n = 3,670$) that survived the RGR screening and had complete trait measurements. Raw trait values are not distributed; the file contains the PCA scores of the log-transformed and standardized traits, the standardized experimental variables, annual RGR and anonymous identifiers for experimental units. `J0E_PCA_loadings.csv` contains the PCA loadings and component standard deviations used in Figure 2.

Column	Description
obs_id	Anonymous seedling identifier
species	Species
block	Abiotic treatment block (1–9; L9 orthogonal design of light, soil moisture and soil phosphorus)
competition	Competition treatment (alone , inter)
pot_id	Anonymous pot identifier
year	Harvest cohort (1st , 2nd , 3rd growing year)
growth_time_z	Standardized seedling age
light_z , moisture_z , soil_P_z	Standardized light, soil moisture and soil phosphorus
RGR	Annual relative height growth rate ($\log \text{ cm yr}^{-1}$)
Above_PC1 – Above_PC3	Aboveground trait axes (leaf economics, plant size, leaf area)
Below_PC1 , Below_PC2	Fine-root trait axes (root collaboration, root economics)

```
public_data <- read.csv("data/J0E_standardized_trait_growth_data.csv")
pca_loadings <- read.csv("data/J0E_PCA_loadings.csv")

# Rename columns to the variable names used throughout the analysis code.
pcatraits_tda <- public_data %>%
  transmute(
    obs_id,
    block = factor(block),
    competition = factor(competition, levels = c("alone", "inter")),
    pot_id = factor(pot_id),
    Nenv = block,
    Light = light_z,
    Moisture = moisture_z,
    AP = soil_P_z,
    Compete = competition,
    sp = factor(.data$species, levels = .env$species),
    growth_time = growth_time_z,
    comp_year = year,
    RGR,
    Above_PC1, Above_PC2, Above_PC3, Below_PC1, Below_PC2
  )

stopifnot(
  nrow(pcatraits_tda) == 3670L,
  nlevels(pcatraits_tda$block) == 9L,
  !anyNA(pcatraits_tda)
)
```

2. PCA trait axes (Figure 2)

```
source(file.path(report_dir, "R", "visualization_corefun_new.R"))
# PCA scores are provided in the public data. Loadings and component standard
# deviations are read from JOE_PCA_loadings.csv (aboveground: princomp on the
# correlation matrix; fine roots: varimax-rotated principal components).
pca_component <- function(pca_name, traits) {
  tab <- pca_loadings[pca_loadings$pca == pca_name, ]
  pcs <- paste0("PC", seq_along(traits))
  loadings <- as.matrix(tab[match(traits, tab$row), pcs])
  dimnames(loadings) <- list(traits, pcs)
  list(loadings = loadings, sd = unlist(tab[tab$row == "SD", pcs]))
}
above_pr <- pca_component("aboveground", above_traits)
froot_pr <- pca_component("fine_root", fineroot_traits)

above_loadings_matrix <- as.matrix(above_pr$loadings)
above_loadings_df <- data.frame(Variable = rownames(above_loadings_matrix), above_loadings_matrix)
above_loadings_df <- above_loadings_df[1:6, ]
above_loadings_df[, -1] <- round(above_loadings_df[, -1], 2)
rownames(above_loadings_df) <- paste0("PC", 1:6)

froot_loadings_matrix <- as.matrix(froot_pr$loadings)
froot_loadings_df <- data.frame(Variable = rownames(froot_loadings_matrix), froot_loadings_matrix)
froot_loadings_df <- froot_loadings_df[1:4, ]
froot_loadings_df[, -1] <- round(froot_loadings_df[, -1], 2)
rownames(froot_loadings_df) <- paste0("PC", 1:4)

cp_shape <- c(21,21,21)

above_pca <- Above_Comp_PCA(pcatraits_tda, above_pr, above_traits,
                           "(a) Aboveground", c(-5.2, 5), c(-5,4.6), cp_shape) +
  theme(legend.position = "none")

above_pca2 <- Above_Comp_PCA2(pcatraits_tda, above_pr, above_traits,
                              "(b) Aboveground", c(-5.2, 5), c(-4, 4.6), cp_shape) +
  theme(legend.position = "none")

froot_pca <- Below_Comp_PCA(pcatraits_tda, froot_pr, fineroot_traits,
                            "(c) Fine roots", c(-4,4.3), c(-5,5), cp_shape) +
  theme(legend.position = "none")

above_pca_plot <- ggMarginal(above_pca, type = "boxplot",
                             groupColour = TRUE, groupFill = FALSE,
                             fill = "white")

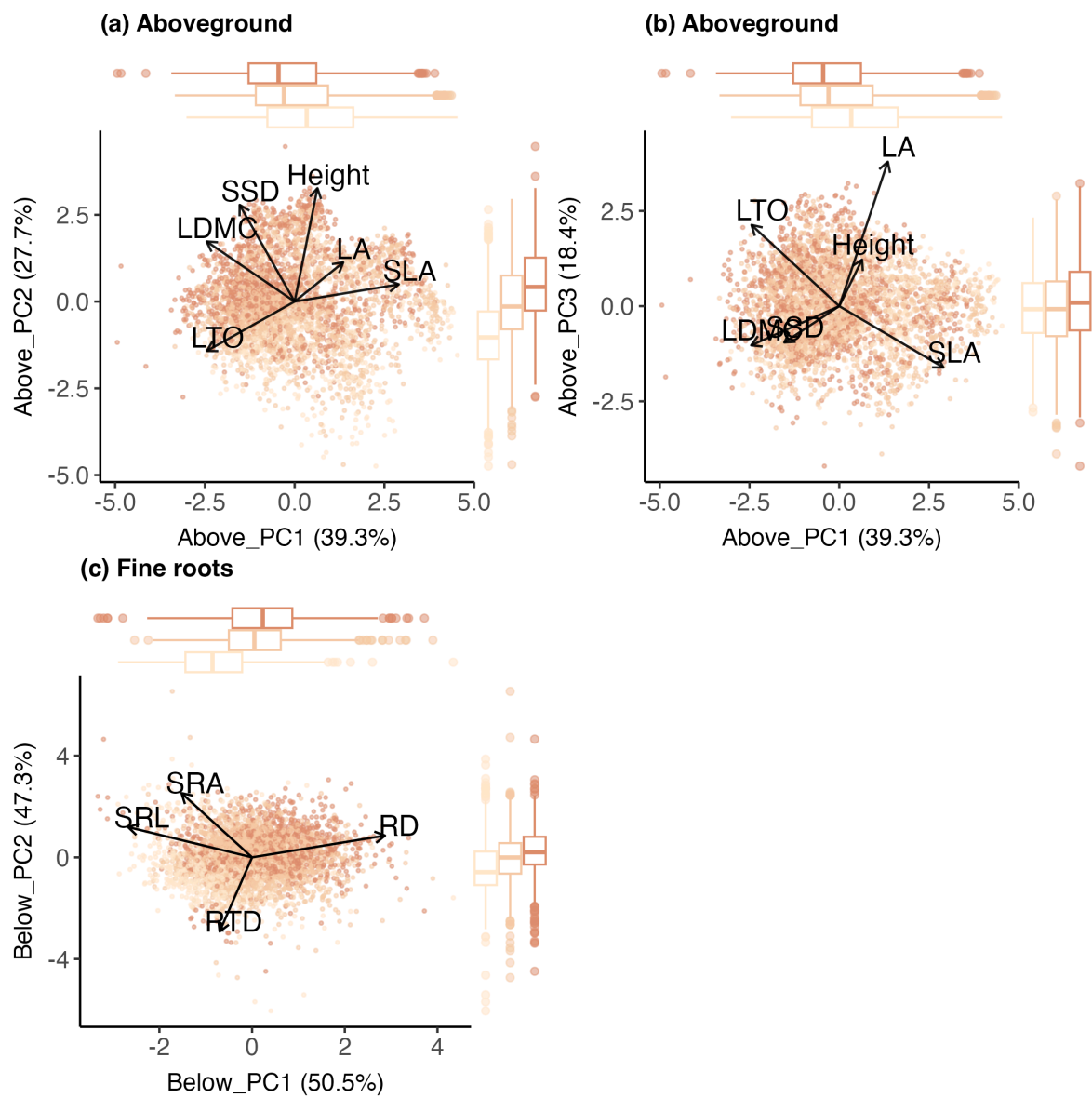
above_pca_plot2 <- ggMarginal(above_pca2, type = "boxplot",
                              groupColour = TRUE, groupFill = FALSE,
                              fill = "white")

froot_pca_plot <- ggMarginal(froot_pca, type = "boxplot",
                             groupColour = TRUE, groupFill = FALSE,
                             fill = "white")

fig3_combined_pca_plot2 <- plot_grid(above_pca_plot, above_pca_plot2, froot_pca_plot,
                                     labels = NULL, ncol = 2)

ggsave(fig3_combined_pca_plot2, file="outputs/figs/fig2_pca_year.pdf", height = 7.5, width = 7.5)
```

Figure 2. Above- and belowground trait axes



3. Year-specific combination-model framework

```
pcatraits_tda1 <- pcatraits_tda
pcatraits_tda1$Above_PC1 <- scale(pcatraits_tda1$Above_PC1)
pcatraits_tda1$Above_PC2 <- scale(pcatraits_tda1$Above_PC2)
pcatraits_tda1$Above_PC3 <- scale(pcatraits_tda1$Above_PC3)
pcatraits_tda1$Below_PC1 <- scale(pcatraits_tda1$Below_PC1)
pcatraits_tda1$Below_PC2 <- scale(pcatraits_tda1$Below_PC2)
pcatraits_tda1$Light <- scale(pcatraits_tda1$Light)
pcatraits_tda1$Moisture <- scale(pcatraits_tda1$Moisture)
pcatraits_tda1$AP <- scale(pcatraits_tda1$AP)
pcatraits_tda1$growth_time <- scale(as.numeric(pcatraits_tda1$growth_time))
pcatraits_tda1$RGR_scale <- scale(pcatraits_tda1$RGR)

tda_1st <- pcatraits_tda1[pcatraits_tda1$comp_year == "1st", ]
tda_2nd <- pcatraits_tda1[pcatraits_tda1$comp_year == "2nd", ]
tda_3rd <- pcatraits_tda1[pcatraits_tda1$comp_year == "3rd", ]
```

Each complete combination model contains one aboveground axis, one belowground axis, one environmental predictor and their trait-trait and trait-environment interactions. The 3 aboveground axes, 2 belowground axes and 4 environmental predictors generate 24 combinations per year.

3.1 Community-level models

```
extract_model_stats <- function(dat, t_a, t_b, e_v) {

  # Complete model within one trait-axis combination (M5)
  f5 <- paste("RGR ~", e_v, "+", t_a, "+", t_b, "+",
             t_a, ":", t_b, "+",
             t_a, ":", e_v, "+",
             t_b, ":", e_v, "+ (1|sp) + (1|block/competition/pot_id)")

  # Fit the model (Removed tryCatch: Errors will stop execution for debugging)
  mod <- lmer(as.formula(f5), data = dat, REML = FALSE,
             control = lmerControl(optimizer = "bobyqa"))

  # --- A. Extract Marginal R2 ---
  r2_out <- MuMIn::r.squaredGLMM(mod)

  if (is.matrix(r2_out)) {
    r2_val <- r2_out[1, "R2m"]
  } else {
    r2_val <- r2_out["R2m"]
  }

  if (is.na(r2_val) || is.nan(r2_val)) r2_val <- NA_real_

  # --- B. Calculate RGR statistics for the data used in model ---
  model_frame <- model.frame(mod)
  mean_rgr <- mean(model_frame[, "RGR"], na.rm = TRUE)
  var_rgr <- var(model_frame[, "RGR"], na.rm = TRUE)

  # --- C. Extract fixed effects coefficients ---
  coefs <- broom.mixed::tidy(mod, effects = "fixed") %>%
    select(term, estimate, std.error, statistic, p.value)

  # Map terms to logical groups
  coefs <- coefs %>%
    mutate(
      Effect_Type = case_when(
        term == "(Intercept)" ~ "Intercept",
        term == e_v ~ "Env",
        term == t_a ~ "Above",
        term == t_b ~ "Below",
        grepl(":", term) & grepl(t_a, term) & grepl(t_b, term) ~ "TT",
        grepl(":", term) & grepl(e_v, term) ~ "TE",
        TRUE ~ "Other"
      )
    )

  # Append model-level statistics
  coefs$Marginal_R2 <- as.numeric(r2_val)
  coefs$Mean_RGR <- mean_rgr
  coefs$Var_RGR <- var_rgr

  return(coefs)
}

above_traits <- c("Above_PC1", "Above_PC2", "Above_PC3")
below_traits <- c("Below_PC1", "Below_PC2")
env_vars <- c("Light", "Moisture", "AP", "Compete")
combinations <- expand.grid(Above = above_traits, Below = below_traits, Env = env_vars, stringsAsFactors = FALSE)
year_datasets <- list("1st" = tda_1st, "2nd" = tda_2nd, "3rd" = tda_3rd)
model_coefficients_results <- data.frame()

for (year_name in names(year_datasets)) {
  current_dat <- year_datasets[[year_name]]

  for (i in 1:nrow(combinations)) {

    curr_above <- combinations$Above[i]
    curr_below <- combinations$Below[i]
    curr_env <- combinations$Env[i]

    res_df <- extract_model_stats(current_dat, curr_above, curr_below, curr_env)

    if (is.null(res_df)) next

    # Add metadata
    res_df$Year <- year_name
    res_df$Combination_ID <- i
    res_df$Trait_Above_Name <- curr_above
    res_df$Trait_Below_Name <- curr_below
  }
}
```

```
    res_df$Env_Var_Name <- curr_env  
    model_coefficients_results <- rbind(model_coefficients_results, res_df)  
  }  
}
```

3.2 Species-level models

```
# 1. Function to extract stats (Species-Specific Model)
extract_species_stats <- function(dat, t_a, t_b, e_v) {

  # --- Sample Size Check ---
  # Remove NAs first to check effective sample size for modeling
  # (Note: lmer will drop NAs automatically, but we check beforehand)
  valid_rows <- sum(!is.na(dat$RGR) & !is.na(dat[[t_a]]) &
    !is.na(dat[[t_b]]) & !is.na(dat[[e_v]]))

  if (valid_rows < 10) return(NULL) # Return NULL to signal skipping

  # Complete model within one species-specific trait-axis combination
  f5 <- paste("RGR ~", e_v, "+", t_a, "+", t_b, "+",
    t_a, ":", t_b, "+",
    t_a, ":", e_v, "+",
    t_b, ":", e_v, "+ (1|block/competition)")

  # Fit the model
  mod <- tryCatch({
    lmer(as.formula(f5), data = dat, REML = FALSE,
      control = lmerControl(optimizer = "bobyqa"))
  }, error = function(e) NULL)

  if (is.null(mod)) return(NULL)

  # --- A. Extract Marginal R2 ---
  r2_out <- tryCatch(MuMIn::r.squaredGLMM(mod), error = function(e) NA)

  if (is.matrix(r2_out)) {
    r2_val <- r2_out[1, "R2m"]
  } else if (!is.na(r2_out)) {
    r2_val <- r2_out["R2m"]
  } else {
    r2_val <- NA_real_
  }

  if (is.na(r2_val) || is.nan(r2_val)) r2_val <- NA_real_

  # --- B. Calculate RGR statistics ---
  model_frame <- model.frame(mod)
  mean_rgr <- mean(model_frame[, "RGR"], na.rm = TRUE)
  var_rgr <- var(model_frame[, "RGR"], na.rm = TRUE)

  # --- C. Extract fixed effects coefficients ---
  coefs <- broom.mixed::tidy(mod, effects = "fixed") %>%
    select(term, estimate, std.error, statistic, p.value)

  # Map terms
  coefs <- coefs %>%
    mutate(
      Effect_Type = case_when(
        term == "(Intercept)" ~ "Intercept",
        term == e_v ~ "Env",
        term == t_a ~ "Above",
        term == t_b ~ "Below",
        grepl(":", term) & grepl(t_a, term) & grepl(t_b, term) ~ "TT",
        grepl(":", term) & grepl(e_v, term) ~ "TE",
        TRUE ~ "Other"
      )
    )

  # Append stats
  coefs$Marginal_R2 <- as.numeric(r2_val)
  coefs$Mean_RGR <- mean_rgr
  coefs$Var_RGR <- var_rgr

  return(coefs)
}

# 2. Prepare Loop Variables
above_traits <- c("Above_PC1", "Above_PC2", "Above_PC3")
below_traits <- c("Below_PC1", "Below_PC2")
env_vars <- c("Light", "Moisture", "AP", "Compete")
combinations <- expand.grid(Above = above_traits, Below = below_traits, Env = env_vars, stringsAsFactors = FALSE)

year_datasets <- list("1st" = tda_1st, "2nd" = tda_2nd, "3rd" = tda_3rd)
species_list <- unique(tda_1st$sp) # Ensure 'sp' column exists

species_results <- data.frame()
```

```

# 3. Main Nested Loop: Year -> Species -> Combination
for (year_name in names(year_datasets)) {
  current_dat <- year_datasets[[year_name]]

  for (sp_name in species_list) {

    # Subset data for current species
    sp_dat <- current_dat[current_dat$sp == sp_name, ]

    # Check total rows before entering combination loop (Optimization)
    if (nrow(sp_dat) < 10) next

    for (i in 1:nrow(combinations)) {

      curr_above <- combinations$Above[i]
      curr_below <- combinations$Below[i]
      curr_env <- combinations$Env[i]

      # Extract stats
      res_df <- extract_species_stats(sp_dat, curr_above, curr_below, curr_env)

      # Handling insufficient data or model failure
      if (is.null(res_df)) {
        # Optional: create a row with NAs if you want to track failures explicitly
        # For now, we just skip
        next
      }

      # Add metadata
      res_df$Year <- year_name
      res_df$Species <- sp_name # <--- New Metadata
      res_df$Combination_ID <- i
      res_df$Trait_Above_Name <- curr_above
      res_df$Trait_Below_Name <- curr_below
      res_df$Env_Var_Name <- curr_env

      species_results <- rbind(species_results, res_df)
    }
  }
}

# 4. Post-processing
head(species_results)

final_species_coefs <- species_results

# Readable aliases for the two primary combination-model result sets.
community_combination_results <- model_coefficients_results
species_combination_results <- final_species_coefs

```

4. H1: Growth predictability

Figure 3 summarizes temporal changes in marginal R-squared at community and species levels.

```

df_community_raw <- model_coefficients_results %>%
  select(Year, Combination_ID, Marginal_R2, Mean_RGR) %>%
  distinct()

df_community_raw1 <- df_community_raw %>%
  mutate(
    Year_num = case_when(
      Year == "1st" ~ 1,
      Year == "2nd" ~ 2,
      Year == "3rd" ~ 3
    )
  )

m_year <- lmer(
  Marginal_R2 ~ Year_num + (1 | Combination_ID),
  data = df_community_raw1
)

df_community_summary <- df_community_raw %>%
  group_by(Year) %>%
  summarise(
    R2 = mean(Marginal_R2, na.rm = TRUE),
    SE = sd(Marginal_R2, na.rm = TRUE) / sqrt(n()),
    RGR = mean(Mean_RGR, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    Year = factor(Year, levels = c("1st", "2nd", "3rd")),
    Level = "Community"
  )

df_species_raw <- final_species_coefs %>%
  select(Year, Species, Combination_ID, Marginal_R2, Mean_RGR) %>%
  distinct()

df_species_raw1 <- df_species_raw %>%
  mutate(
    Year_num = case_when(
      Year == "1st" ~ 1,
      Year == "2nd" ~ 2,
      Year == "3rd" ~ 3,
      TRUE ~ NA_real_
    ),
    Species = factor(Species),
    Combination_ID = factor(Combination_ID)
  )

m_sp2 <- lmer(
  Marginal_R2 ~ Year_num + (1 | Species) + (1 | Combination_ID),
  data = df_species_raw1,
  REML = TRUE
)

df_species_summary <- df_species_raw %>%
  group_by(Year, Species) %>%
  summarise(
    R2 = mean(Marginal_R2, na.rm = TRUE),
    # Calculate the SE across the 24 model combinations.
    # SE = SD(marginal R-squared) / sqrt(24).
    SE = sd(Marginal_R2, na.rm = TRUE) / sqrt(n_distinct(Combination_ID)),
    RGR = mean(Mean_RGR, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    Year = factor(Year, levels = c("1st", "2nd", "3rd")),
    Level = "Species"
  )

df_combined <- bind_rows(
  df_community_summary %>% select(Year, R2, SE, RGR, Level),
  df_species_summary %>% select(Year, R2, SE, RGR, Level)
) %>%
  mutate(Level = factor(Level, levels = c("Community", "Species")))

p_a <- ggplot(df_community_summary, aes(x = Year, y = R2)) +
  geom_errorbar(aes(ymin = R2 - SE, ymax = R2 + SE, color = Year),
    width = 0.15, size = 1) +
  geom_point(aes(color = Year), size = 3) +
  geom_text(aes(label = paste0("Mean RGR: ", sprintf("%.2f", RGR))),

```

```

        color = "black",
        vjust = -2.0,
        hjust = 0.5,
        size = 3.5, fontface = "bold") +
scale_color_manual(values = c("1st"="#FFE7CC", "2nd"="#F8CBA6", "3rd"="#E08E6D")) +
labs(
  title = "(a) Community-level",
  y = expression("Marginal " * italic(R)^2 * "of growth models"),
  x = "Year (seedling age)"
) +
scale_y_continuous(expand = expansion(mult = c(0.1, 0.2))) +
theme_Publication(base_size = 11) +
theme(legend.position = "none")

p_b <- ggplot(df_species_summary, aes(x = Year, y = R2)) +
  geom_violin(aes(color = Year), fill = NA, trim = FALSE, size = 0.8) +
  geom_jitter(aes(color = Year), shape = 17, size = 2.5, alpha = 0.8, width = 0.1) +
  scale_color_manual(values = c("1st"="#FFE7CC", "2nd"="#F8CBA6", "3rd"="#E08E6D")) +
  labs(
    title = "(b) Species-level",
    y = NULL,
    x = "Year (seedling age)"
  ) +
  theme_Publication(base_size = 11) +
  theme(legend.position = "none")

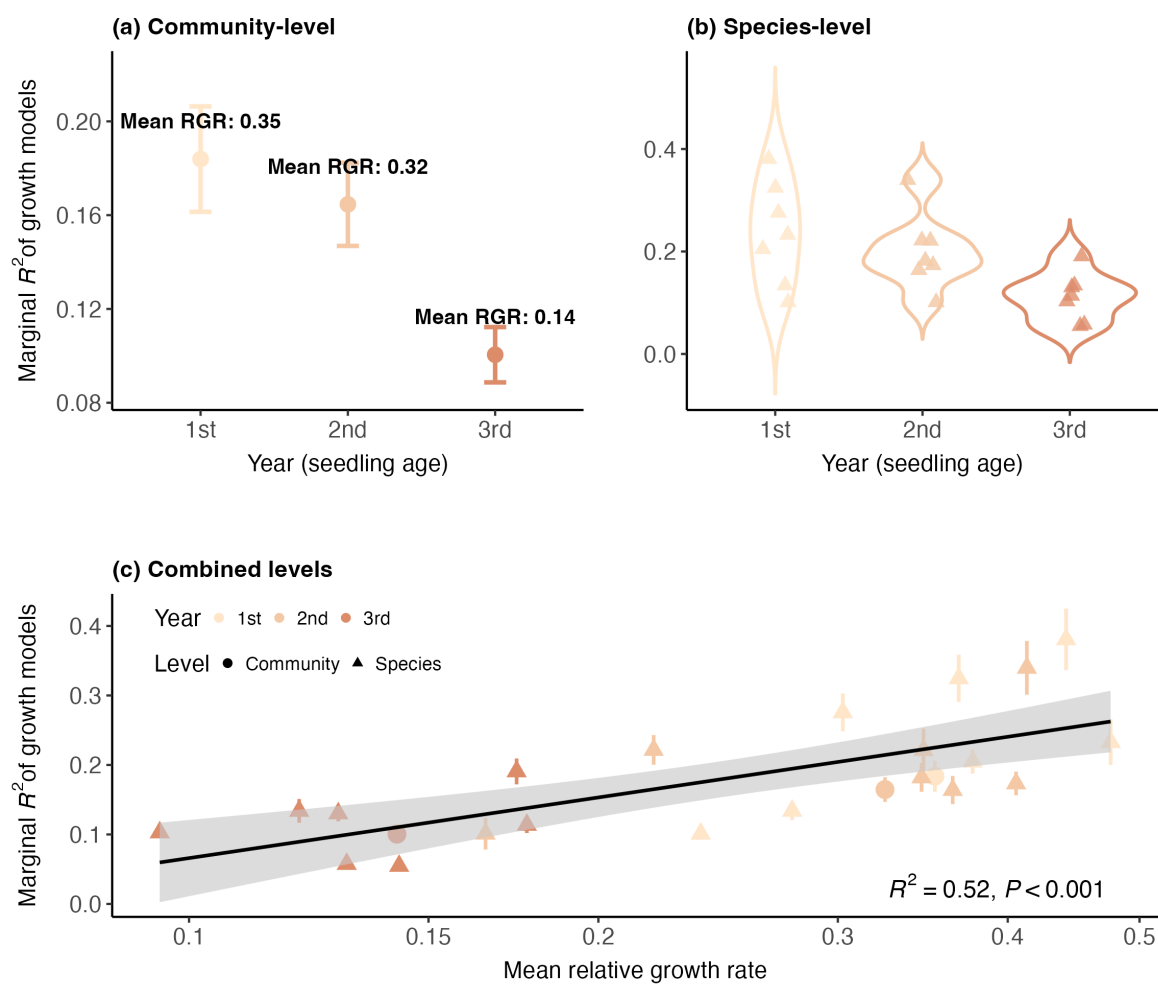
p_c <- ggplot(df_combined, aes(x = RGR, y = R2)) +
  geom_errorbar(aes(ymin = R2 - SE, ymax = R2 + SE, color = Year),
    width = 0, size = 0.8, alpha = 1) +
  geom_point(aes(color = Year, shape = Level, size = Level), alpha = 95) +
  geom_smooth(method = "lm", color = "black", se = TRUE, size = 0.8, fill = "grey80", alpha = 0.6) +
  stat_poly_eq(
    aes(label = paste(after_stat(rr.label), after_stat(p.value.label), sep = "*\\", "\\*")),
    formula = y ~ x, parse = TRUE,
    label.x = "right", label.y = "bottom", size = 4
  ) +
  scale_color_manual(
    values = c("1st"="#FFE7CC", "2nd"="#F8CBA6", "3rd"="#E08E6D"),
    name = "Year"
  ) +
  scale_shape_manual(
    values = c("Community" = 19, "Species" = 17),
    name = "Level"
  ) +
  scale_size_manual(
    values = c("Community" = 3.5, "Species" = 3),
    guide = "none"
  ) +
  scale_x_log10(
    breaks = c(0.1, 0.15, 0.2, 0.3, 0.4, 0.5),
    labels = scales::label_number(drop0trailing = TRUE)
  ) +
  labs(
    title = "(c) Combined levels",
    y = expression("Marginal " * italic(R)^2 * "of growth models"),
    x = "Mean relative growth rate"
  ) +
  guides(
    color = guide_legend(order = 1, nrow = 1),
    shape = guide_legend(order = 2, nrow = 1)
  ) +
  theme_Publication(base_size = 11) +
  theme(
    legend.position = c(0.04, 0.96),
    legend.justification = c(0, 1),
    legend.box = "vertical",
    legend.direction = "horizontal",
    legend.background = element_blank()
  )

final_plot <- (p_a + p_b) / p_c + plot_layout(heights = c(1, 0.9))

ggsave(
  final_plot <- (p_a + p_b) / p_c + plot_layout(heights = c(1, 0.9)),
  file = "outputs/figs/fig3_part_growth_r2.pdf",
  height = 7.5, width = 7
)

```

Figure 3. Temporal change in growth predictability



5. H2-H3: Relative contributions

The same year-specific combination models are used to test both hypotheses. H2 concerns the reorganization of trait main effects, above- and belowground contributions and trait-trait interactions. H3 concerns temporal changes in environmental main effects and trait-environment interactions. These results are presented together in Figure 4.

Within every trait-axis combination, marginal R-squared is decomposed sequentially across five nested models: environment, aboveground trait, belowground trait, trait-trait interaction and the two trait-environment interactions entered together. For predictor-specific summaries, the combined trait-environment contribution is divided equally between the aboveground-trait $\times$ environment and belowground-trait $\times$ environment terms. This retains the original reduced-model calculation and avoids fitting additional entry-order models.

Community-level decomposition.

```

decompose_triplet_community <- function(dat, t_a, t_b, e_v) {
  default_res <- c(Env=0, Above=0, Below=0, TT=0, TE=0)

  f1 <- paste("RGR ~", e_v, "+ (1|sp) + (1|block/competition/pot_id)")
  f2 <- paste("RGR ~", e_v, "+", t_a, "+ (1|sp) + (1|block/competition/pot_id)")
  f3 <- paste("RGR ~", e_v, "+", t_a, "+", t_b, "+ (1|sp) + (1|block/competition/pot_id)")
  f4 <- paste("RGR ~", e_v, "+", t_a, "+", t_b, "+", t_a,":",t_b, "+ (1|sp) + (1|block/competition/pot_id)")
  f5 <- paste("RGR ~", e_v, "+", t_a, "+", t_b, "+", t_a,":",t_b,
    "+", t_a,":",e_v, "+", t_b,":",e_v,
    "+ (1|sp) + (1|block/competition/pot_id)")

  get_r2 <- function(f) {
    mod <- tryCatch({
      lmer(as.formula(f), data = dat, REML = FALSE, control = lmerControl(optimizer = "bobyqa"))
    }, error = function(e) NULL, warning = function(w) NULL)

    if (is.null(mod)) return(0)

    val <- tryCatch({
      r2_out <- MuMIn::r.squaredGLMM(mod)
      v <- if (is.matrix(r2_out)) r2_out[1, "R2m"] else r2_out["R2m"]
      if (is.na(v) || is.nan(v)) 0 else as.numeric(v)
    }, error = function(e) 0)
    return(val)
  }

  r2_values <- tryCatch({
    c(M1=get_r2(f1), M2=get_r2(f2), M3=get_r2(f3), M4=get_r2(f4),
      M5=get_r2(f5))
  }, error = function(e) NULL)

  if (is.null(r2_values) || length(r2_values) != 5 || any(is.na(r2_values))) return(default_res)

  r2_values <- cummax(r2_values)
  total_r2 <- unname(r2_values["M5"])

  if (total_r2 < 0.0001) return(default_res)

  incr <- c(
    Env = unname(r2_values["M1"]),
    Above = unname(r2_values["M2"] - r2_values["M1"]),
    Below = unname(r2_values["M3"] - r2_values["M2"]),
    TT = unname(r2_values["M4"] - r2_values["M3"]),
    TE = unname(r2_values["M5"] - r2_values["M4"])
  )
  incr[incr < 0] <- 0
  return(incr / total_r2)
}

above_traits <- c("Above_PC1", "Above_PC2", "Above_PC3")
below_traits <- c("Below_PC1", "Below_PC2")
env_vars <- c("Light", "Moisture", "AP", "Compete")
combinations <- expand.grid(Above = above_traits, Below = below_traits, Env = env_vars, stringsAsFactors = FALSE)

year_datasets <- list("1st" = tda_1st, "2nd" = tda_2nd, "3rd" = tda_3rd)

comm_raw_results <- data.frame()

for (year_name in names(year_datasets)) {
  current_dat <- year_datasets[[year_name]]

  for (i in 1:nrow(combinations)) {
    curr_above <- combinations$Above[i]
    curr_below <- combinations$Below[i]
    curr_env <- combinations$Env[i]

    contribs <- decompose_triplet_community(current_dat, curr_above, curr_below, curr_env)

    if (is.null(contribs) || length(contribs) != 5 || any(is.na(contribs))) {
      contribs <- c(Env=0, Above=0, Below=0, TT=0, TE=0)
    }

    tmp_raw <- data.frame(
      Year = rep(year_name, 5),
      Combination_ID = rep(i, 5),
      Trait_Above = rep(curr_above, 5),
      Trait_Below = rep(curr_below, 5),
      Env_Var = rep(curr_env, 5),
      Component = c("Env", "Above", "Below", "TT", "TE"),
      Value = as.numeric(contribs),
      stringsAsFactors = FALSE
    )
  }
}

```

```

    )
    comm_raw_results <- rbind(comm_raw_results, tmp_raw)
  }
}

df_comm <- comm_raw_results %>%
  mutate(
    Level = "Community",
    Component = factor(Component, levels = c("Env", "Above", "Below", "TT", "TE")),
    Component_Label = case_when(
      Component == "Env" ~ "Environment",
      Component == "Above" ~ "Above-ground",
      Component == "Below" ~ "Below-ground",
      Component == "TT" ~ "Trait*Trait",
      Component == "TE" ~ "Trait*Env",
      TRUE ~ as.character(Component)
    ),
    Component_Label = factor(
      Component_Label,
      levels = c("Environment", "Above-ground", "Below-ground", "Trait*Trait", "Trait*Env")
    ),
    Year_Num = case_when(
      Year == "1st" ~ 1,
      Year == "2nd" ~ 2,
      Year == "3rd" ~ 3
    ),
    Combination_ID = factor(Combination_ID)
  ) %>%
  select(Year, Year_Num, Level, Combination_ID, Component, Component_Label, Value)

str(df_comm)
head(df_comm)
df_comm <- comm_raw_results %>%
  mutate(
    Combination_ID = factor(Combination_ID),
    Component = factor(Component, levels = c("Env", "Above", "Below", "TT", "TE")),
    Component_Label = case_when(
      Component == "Env" ~ "Environment",
      Component == "Above" ~ "Above-ground",
      Component == "Below" ~ "Below-ground",
      Component == "TT" ~ "Trait*Trait",
      Component == "TE" ~ "Trait*Env",
      TRUE ~ as.character(Component)
    ),
    Component_Label = factor(Component_Label,
      levels = c("Environment", "Above-ground", "Below-ground", "Trait*Trait", "Trait*Env")
    ),
    Year_Num = case_when(
      Year == "1st" ~ 1,
      Year == "2nd" ~ 2,
      Year == "3rd" ~ 3
    )
  ) %>%
  select(Year, Year_Num, Combination_ID, Component, Component_Label, Value)

fit_comm_component <- function(dat) {
  m <- lmer(Value ~ Year_Num + (1 | Combination_ID), data = dat, REML = TRUE)
  co <- summary(m)$coefficients["Year_Num", ]
  r2 <- r.squaredGLMM(m)[1] # marginal R2

  p <- unname(co["Pr(>|t|)"])

  tibble(
    slope = unname(co["Estimate"]),
    se = unname(co["Std. Error"]),
    p = p,
    star = case_when(
      p < 0.001 ~ "***",
      p < 0.01 ~ "**",
      p < 0.05 ~ "*",
      p < 0.1 ~ ".",
      TRUE ~ ""
    ),
    R2 = r2
  )
}

res_comm_component <- df_comm %>%
  group_by(Component, Component_Label) %>%
  group_modify(~ fit_comm_component(.x)) %>%
  ungroup()

```

```

slopes_comm <- res_comm_component %>%
  mutate(
    star = ifelse(!is.na(star), star,
      case_when(
        p < 0.001 ~ "***",
        p < 0.01 ~ "**",
        p < 0.05 ~ "*",
        p < 0.1 ~ ".",
        TRUE ~ ""
      )
    ),
    lab = paste0("b=", formatC(slope, format = "f", digits = 3), star),
    x_pos = 3.15
  )

ggplot(df_comm, aes(x = Year_Num, y = Value, color = Component_Label, fill = Component_Label)) +
  geom_smooth(aes(group = Component_Label), method = "lm", se = TRUE, linewidth = 1, alpha = 0.3) +
  scale_color_manual(values = my_colors) +
  scale_fill_manual(values = my_colors) +
  scale_x_continuous(breaks = 1:3, labels = c("1st", "2nd", "3rd")) +
  labs(title = "(c) Community-level", y = NULL, x = "Year") +
  theme_Publication()

```

```

df_env <- comm_raw_results %>%
  filter(Component == "Env") %>%
  mutate(Specific_Var = Env_Var, Analysis_Group = "Env_Main")

# 1.2 Main Effect (Traits)
df_trait <- comm_raw_results %>%
  filter(Component %in% c("Above", "Below")) %>%
  mutate(
    Specific_Var = ifelse(Component=="Above", Trait_Above, Trait_Below),
    Analysis_Group = "Trait_Main"
  )

# 1.3 TT Interaction (Split to Traits)
df_tt <- comm_raw_results %>%
  filter(Component == "TT") %>%
  pivot_longer(cols = c(Trait_Above, Trait_Below), values_to = "Specific_Var") %>%
  mutate(Value = Value / 2, Analysis_Group = "TT_Interaction")

# 1.4 TE Interaction (Split to Traits)
df_te_trait <- comm_raw_results %>%
  filter(Component == "TE") %>%
  pivot_longer(cols = c(Trait_Above, Trait_Below), values_to = "Specific_Var") %>%
  mutate(Value = Value / 2, Analysis_Group = "TE_Trait")

# 1.5 TE Interaction (Grouped by Environment)
df_te_env <- comm_raw_results %>%
  filter(Component == "TE") %>%
  mutate(
    Specific_Var = Env_Var,
    Value = Value / 2,
    Analysis_Group = "TE_Env"
  )

# Combine the component-specific results.
df_all_analysis <- bind_rows(
  df_env %>% select(Year, Combination_ID, Value, Specific_Var, Analysis_Group),
  df_trait %>% select(Year, Combination_ID, Value, Specific_Var, Analysis_Group),
  df_tt %>% select(Year, Combination_ID, Value, Specific_Var, Analysis_Group),
  df_te_trait %>% select(Year, Combination_ID, Value, Specific_Var, Analysis_Group),
  df_te_env %>% select(Year, Combination_ID, Value, Specific_Var, Analysis_Group)
) %>%
  mutate(
    Year_Num = case_when(Year == "1st" ~ 1, Year == "2nd" ~ 2, Year == "3rd" ~ 3),
    Combination_ID = as.factor(Combination_ID),
    Value = pmax(0, Value) # Prevent negative incremental contributions caused by numerical imprecision.
  )

# 2. Temporal tests for all five contribution groups
run_trend_test_complete <- function(data) {
  unique_groups <- unique(data$Analysis_Group)
  results_list <- list()

  for (grp in unique_groups) {
    sub_data <- data %>% filter(Analysis_Group == grp)
    unique_vars <- unique(sub_data$Specific_Var)

    for (var in unique_vars) {
      var_data <- sub_data %>% filter(Specific_Var == var)

      if (var(var_data$Value, na.rm=TRUE) > 1e-10) {
        mod <- tryCatch({
          lmer(Value ~ Year_Num + (1 | Combination_ID), data = var_data)
        }, error = function(e) NULL)

        sig_mark <- ""
        if (!is.null(mod)) {
          res <- broom.mixed::tidy(mod, effects = "fixed") %>% filter(term == "Year_Num")
          if (nrow(res) > 0 && !is.na(res$p.value)) {
            p <- res$p.value
            if (p < 0.001) sig_mark <- "***"
            else if (p < 0.01) sig_mark <- "**"
            else if (p < 0.05) sig_mark <- "*"
            else if (p < 0.1) sig_mark <- "."
          }
        }

        results_list[[paste(grp, var, sep="_")] <- data.frame(
          Analysis_Group = grp, Specific_Var = var, Label = sig_mark, stringsAsFactors = FALSE
        )
      }
    }
  }
}

```

```

    }
    return(do.call(rbind, results_list))
  }

sig_results_complete <- run_trend_test_complete(df_all_analysis)

# 3. Prepare plotting data
plot_data_agg <- df_all_analysis %>%
  group_by(Year, Analysis_Group, Specific_Var) %>%
  summarise(
    Mean = mean(Value, na.rm=TRUE),
    SE = sd(Value, na.rm=TRUE)/sqrt(n()),
    .groups = "drop"
  )

### Test temporal trends for individual predictors
df_all_analysis2 <- df_all_analysis %>%
  mutate(
    Year = factor(Year, levels = c("1st", "2nd", "3rd")),
    Year_Num = as.numeric(Year_Num),
    Combination_ID = factor(Combination_ID),
    Analysis_Group = factor(Analysis_Group),
    Specific_Var = factor(Specific_Var)
  )

# 2. Fit a temporal model for each contribution group and predictor.
fit_one <- function(dat) {
  # Return missing values when a subset has insufficient observations or years.
  if (nrow(dat) < 6 || length(unique(dat$Year_Num)) < 2) {
    return(tibble(
      slope = NA_real_, se = NA_real_, df = NA_real_,
      t = NA_real_, p = NA_real_, ci_low = NA_real_, ci_high = NA_real_
    ))
  }

  m <- lmer(Value ~ Year_Num + (1 | Combination_ID), data = dat, REML = TRUE)

  co <- summary(m)$coefficients
  slope <- co["Year_Num", "Estimate"]
  se <- co["Year_Num", "Std. Error"]
  df <- co["Year_Num", "df"]
  tval <- co["Year_Num", "t value"]
  pval <- co["Year_Num", "Pr(>|t|)"]

  # Use Wald 95% confidence intervals for the temporal slopes.
  ci <- suppressMessages(confint(m, parm = "Year_Num", method = "Wald"))

  tibble(
    slope = slope, se = se, df = df, t = tval, p = pval,
    ci_low = ci[1], ci_high = ci[2]
  )
}

# 3. Fit all models and return a tidy result object.
df_all_analysis2 <- df_all_analysis %>%
  mutate(
    Year_Num = as.numeric(Year_Num),
    Combination_ID = factor(Combination_ID),
    Analysis_Group = factor(Analysis_Group),
    Specific_Var = factor(Specific_Var)
  )

# 2. Fit one subset and extract the required statistics.
fit_extract <- function(dat) {
  # Skip subsets with insufficient data.
  if (nrow(dat) < 6 || length(unique(dat$Year_Num)) < 2) {
    return(tibble(
      slope = NA_real_,
      se = NA_real_,
      p = NA_real_,
      star = NA_character_,
      R2 = NA_real_
    ))
  }

  m <- lmer(Value ~ Year_Num + (1 | Combination_ID),
    data = dat, REML = TRUE)

  co <- summary(m)$coefficients["Year_Num", ]

```

```

slope <- co["Estimate"]
se <- co["Std. Error"]
pval <- co["Pr(>|t|)"]

# Significance label.
star <- case_when(
  pval < 0.001 ~ "***",
  pval < 0.01 ~ "**",
  pval < 0.05 ~ "*",
  TRUE ~ ""
)

# Marginal R-squared for the fixed effects.
R2m <- r.squaredGLMM(m)[1]

tibble(
  slope = slope,
  se = se,
  p = pval,
  star = star,
  R2 = R2m
)
}

# 3. Fit every group-by-predictor subset.
res_trend_minimal <- df_all_analysis2 %>%
  group_by(Analysis_Group, Specific_Var) %>%
  group_modify(~ fit_extract(.x)) %>%
  ungroup() %>%
  arrange(Analysis_Group, p)

## Test temporal trends by predictor group
df0 <- df_all_analysis %>%
  mutate(
    Year_Num = as.numeric(Year_Num),
    Combination_ID = factor(Combination_ID),
    Specific_Var = factor(Specific_Var),
    Analysis_Group = factor(Analysis_Group)
  )

fit_common_trend <- function(dat) {
  m <- lmer(Value ~ Year_Num + (1 | Combination_ID) + (1 | Specific_Var),
    data = dat, REML = TRUE)

  co <- summary(m)$coefficients["Year_Num", ]
  out <- data.frame(
    slope = unname(co["Estimate"]),
    se = unname(co["Std. Error"]),
    p = unname(co["Pr(>|t|)"])
  )
  out$star <- ifelse(out$p < 0.001, "***",
    ifelse(out$p < 0.01, "**",
      ifelse(out$p < 0.05, "*", "")))
  return(list(model = m, stats = out))
}

dat_above <- df0 %>%
  filter(Analysis_Group == "Trait_Main",
    Specific_Var %in% c("Above_PC1", "Above_PC2", "Above_PC3"))

res_above <- fit_common_trend(dat_above)

res_above$stats # slope / se / p / star

dat_below <- df0 %>%
  filter(Analysis_Group == "Trait_Main",
    Specific_Var %in% c("Below_PC1", "Below_PC2"))

res_below <- fit_common_trend(dat_below)

res_below$stats

dat_env3 <- df0 %>%
  filter(Analysis_Group == "Env_Main",
    Specific_Var %in% c("Light", "Moisture", "AP"))

res_env3 <- fit_common_trend(dat_env3)
res_env3$stats

dat_tt_above <- df0 %>%

```

```

filter(
  Analysis_Group == "TT_Interaction",
  Specific_Var %in% c(
    "Above_PC1",
    "Above_PC2",
    "Above_PC3",
    "Below_PC1",
    "Below_PC2"
  )
)
m_tt_above <- lmer(
  Value ~ Year_Num + (1 | Combination_ID) + (1|Specific_Var),
  data = dat_tt_above,
  REML = TRUE
)

# Prepare labels.
label_data <- plot_data_agg %>%
  group_by(Analysis_Group, Specific_Var) %>%
  summarise(Max_Y = max(Mean + SE, na.rm=TRUE), .groups="drop") %>%
  left_join(sig_results_complete, by = c("Analysis_Group", "Specific_Var")) %>%
  mutate(Year = "2nd", Label = ifelse(is.na(Label), "", Label))

# 4. Plotting functions and final assembly
plot_group <- function(grp_name, title, remove_x=FALSE) {
  d <- plot_data_agg %>% filter(Analysis_Group == grp_name)
  l <- label_data %>% filter(Analysis_Group == grp_name)

  # Preserve the intended environmental-predictor order.
  if (grepl("Env", grp_name)) {
    levs <- c("Light", "Moisture", "AP", "Compete")
    d$Specific_Var <- factor(d$Specific_Var, levels = levs)
    l$Specific_Var <- factor(l$Specific_Var, levels = levs)
  }

  p <- ggplot(d, aes(x = Specific_Var, y = Mean, color = Year, group = Year)) +
    geom_errorbar(aes(ymin = pmax(0, Mean - SE), ymax = Mean + SE),
      width = 0.2, position = position_dodge(width = 0.6)) +
    geom_point(position = position_dodge(width = 0.6), size = 3) +
    geom_text(data = l, aes(x = Specific_Var, y = Max_Y * 1.15, label = Label),
      inherit.aes = FALSE, size = 5, color = "black", fontface = "bold") +
    scale_color_manual(values = c("1st"="#FFE7CC", "2nd"="#F8CBA6", "3rd"="#E08E6D")) +
    labs(
      title = title,
      x = NULL,
      y = "Relative importance of predictors in explaining growth variation",
      color = "Year"
    ) +
    theme_Publication() +
    theme(legend.position = "none")

  if (remove_x) p <- p + theme(axis.text.x = element_blank())
  else p <- p + theme(axis.text.x = element_text(angle=30, hjust=1, color="black"))

  return(p)
}

## =====
## Build panels (a-e) with full x-axis labels for traits and envs
## =====

trait_x_labels <- c(
  "Above_PC1" = "Leaf economics\n(Above_PC1)",
  "Above_PC2" = "Plant size\n(Above_PC2)",
  "Above_PC3" = "Leaf area\n(Above_PC3)",
  "Below_PC1" = "Root collaboration\n(Below_PC1)",
  "Below_PC2" = "Root economics\n(Below_PC2)"
)

env_x_labels <- c(
  "Light" = "Light",
  "Moisture" = "Soil moisture",
  "AP" = "Soil phosphorus",
  "Compete" = "Competition"
)

## ---- 2) Apply x-axis helper functions ----
apply_trait_x_axis <- function(p, angle = 35, hjust = 1, vjust = 1, size = 9.5) {
  p +
    scale_x_discrete(labels = trait_x_labels) +
    theme(

```

```

    axis.text.x = element_text(
      angle = angle, hjust = hjust, vjust = vjust,
      size = size, colour = "black"
    )
  )
}

apply_env_x_axis <- function(p, angle = 0, hjust = 0.5, vjust = 1, size = 9.5) {
  p +
    scale_x_discrete(labels = env_x_labels) +
    theme(
      axis.text.x = element_text(
        angle = angle, hjust = hjust, vjust = vjust,
        size = size, colour = "black"
      )
    )
}

## ---- 3) Common margins (avoid repetition) ----
panel_margin <- theme(plot.margin = margin(t = 1, r = 1, b = 4, l = 1, unit = "pt"))

## ---- 4) Generate panels ----
p_a <- plot_group("Trait_Main", "(a) Trait main effect", remove_x = TRUE) +
  ylab(NULL) + panel_margin

p_b <- plot_group("TT_Interaction", "(b) Trait × Trait interaction", remove_x = TRUE) +
  panel_margin

## (c) x = trait -> full trait labels
p_c <- plot_group("TE_Trait", "(c) Trait × Env interaction (by Trait)", remove_x = FALSE) +
  ylab(NULL) + panel_margin
p_c <- apply_trait_x_axis(p_c, angle = 35, hjust = 0.8, size = 9.5)

p_d <- plot_group("Env_Main", "(d) Environment main effect", remove_x = TRUE) +
  ylab(NULL) + panel_margin

## (e) x = env -> full env labels
p_e <- plot_group("TE_Env", "(e) Trait × Env interaction (by Env)", remove_x = FALSE) +
  ylab(NULL) + panel_margin
p_e <- apply_env_x_axis(p_e, angle = 15, hjust = 0.8, size = 9.5)

# Assemble the final panels.
fig4a_e <- (p_a / p_b / p_c) | (p_d / p_e + plot_layout(heights = c(1, 1)))
fig4a_e <- fig4a_e + plot_layout(widths = c(1.3, 1), guides = "collect") & theme(legend.position = "bottom")

```

Species-level decomposition.

```

# 1. Define the species-level decomposition function
decompose_triplet <- function(dat, t_a, t_b, e_v) {
  # Use the original species-level model sequence.
  # Use the updated block-by-competition hierarchy for species-level models.
  f1 <- paste("RGR_scale ~", e_v, "+ (1|block/competition)")
  f2 <- paste("RGR_scale ~", e_v, "+", t_a, "+ (1|block/competition)")
  f3 <- paste("RGR_scale ~", e_v, "+", t_a, "+", t_b, "+ (1|block/competition)")
  f4 <- paste("RGR_scale ~", e_v, "+", t_a, "+", t_b, "+", t_a, ":", t_b, "+ (1|block/competition)")
  f5 <- paste("RGR_scale ~", e_v, "+", t_a, "+", t_b, "+", t_a, ":", t_b,
    "+", t_a, ":", e_v, "+", t_b, ":", e_v,
    "+ (1|block/competition)")

  get_r2 <- function(f) {
    mod <- tryCatch({
      lmer(as.formula(f), data = dat, REML = FALSE, control = lmerControl(optimizer = "bobyqa"))
    }, error = function(e) NULL, warning = function(w) NULL)

    if (is.null(mod)) return(0)

    val <- tryCatch({
      r2_out <- MuMIn::r.squaredGLMM(mod)
      if (is.matrix(r2_out)) r2_out[1, "R2m"] else r2_out["R2m"]
    }, error = function(e) 0)
    return(if (is.na(val)) 0 else as.numeric(val))
  }

  r2_values <- c(
    M1=get_r2(f1), M2=get_r2(f2), M3=get_r2(f3), M4=get_r2(f4),
    M5=get_r2(f5)
  )
  r2_values[is.na(r2_values)] <- 0
  r2_values <- cummax(r2_values)

  total_r2 <- unname(r2_values["M5"])
  if(is.na(total_r2) || total_r2 < 0.0001) {
    return(c(Env=0, Above=0, Below=0, TT=0, TE=0))
  }

  incr <- c(
    Env = unname(r2_values["M1"]),
    Above = unname(r2_values["M2"] - r2_values["M1"]),
    Below = unname(r2_values["M3"] - r2_values["M2"]),
    TT = unname(r2_values["M4"] - r2_values["M3"]),
    TE = unname(r2_values["M5"] - r2_values["M4"])
  )
  incr[incr < 0] <- 0 # Preserve names while constraining contributions to be non-negative.
  return(incr)
}

# 2. Define loop variables
# Use standardized RGR as the response.
above_traits <- c("Above_PC1", "Above_PC2", "Above_PC3")
below_traits <- c("Below_PC1", "Below_PC2")
env_vars <- c("Light", "Moisture", "AP", "Compete") # Competition predictor.
combinations <- expand.grid(Above = above_traits, Below = below_traits, Env = env_vars, stringsAsFactors = FALSE)

year_datasets <- list("1st" = tda_1st, "2nd" = tda_2nd, "3rd" = tda_3rd)
species_list <- unique(tda_1st$sp)

results_list <- list()
counter <- 1

for (year_name in names(year_datasets)) {
  current_dat <- year_datasets[[year_name]]

  for (sp_name in species_list) {
    sp_dat <- current_dat[current_dat$sp == sp_name, ]

    if (nrow(sp_dat) < 15) next
    for (i in 1:nrow(combinations)) {
      raw_incr <- decompose_triplet(sp_dat, combinations$Above[i], combinations$Below[i], combinations$Env[i])
      total_r2 <- sum(raw_incr, na.rm = TRUE)

      if(total_r2 > 0.001) {
        rel_contrib <- raw_incr / total_r2
        results_list[[counter]] <- data.frame(
          Year = year_name,
          Species = sp_name,
          Combination_ID = i,
          Trait_Above = combinations$Above[i],

```

```

        Trait_Below = combinations$Below[i],
        Env_Var = combinations$Env[i],
        Val_Env = unname(rel_contrib["Env"]),
        Val_Above = unname(rel_contrib["Above"]),
        Val_Below = unname(rel_contrib["Below"]),
        Val_TT = unname(rel_contrib["TT"]),
        Val_TE = unname(rel_contrib["TE"]),
        Total_R2 = total_r2,
        stringsAsFactors = FALSE
      )
      counter <- counter + 1
    }
  }
}
species_raw_detailed <- bind_rows(results_list)

df_community <- comm_raw_results %>%
  mutate(
    Species = "Community",
    Level = "Community") %>%
  select(Year, Level, Species, Combination_ID, Component, Value)%>%
  mutate(
    Component_Label = case_when(
      Component == "Env" ~ "Environment",
      Component == "Above" ~ "Above-ground",
      Component == "Below" ~ "Below-ground",
      Component == "TT" ~ "Trait*Trait",
      Component == "TE" ~ "Trait*Env",
      TRUE ~ Component
    ),
    Component_Label = factor(Component_Label,
      levels = c("Environment", "Above-ground", "Below-ground", "Trait*Trait", "Trait*Env")),
    # Convert harvest year to a numeric variable.
    Year_Num = case_when(
      Year == "1st" ~ 1,
      Year == "2nd" ~ 2,
      Year == "3rd" ~ 3
    )
  )

df_species <- species_raw_detailed %>%
  select(Year, Species, Combination_ID, Val_Env, Val_Above, Val_Below, Val_TT, Val_TE) %>%
  pivot_longer(
    cols = starts_with("Val_"),
    names_to = "Component",
    values_to = "Value"
  ) %>%
  mutate(
    Level = "Species",
    Component = recode(Component,
      "Val_Env" = "Env",
      "Val_Above" = "Above",
      "Val_Below" = "Below",
      "Val_TT" = "TT",
      "Val_TE" = "TE"
    )
  ) %>%
  select(Year, Level, Species, Combination_ID, Component, Value)%>%
  mutate(
    # Assign descriptive component labels.
    Component_Label = case_when(
      Component == "Env" ~ "Environment",
      Component == "Above" ~ "Above-ground",
      Component == "Below" ~ "Below-ground",
      Component == "TT" ~ "Trait*Trait",
      Component == "TE" ~ "Trait*Env",
      TRUE ~ Component
    ),
    # Set the plotting order.
    Component_Label = factor(Component_Label,
      levels = c("Environment", "Above-ground", "Below-ground", "Trait*Trait", "Trait*Env")),
    # Convert harvest year to a numeric variable.
    Year_Num = case_when(
      Year == "1st" ~ 1,
      Year == "2nd" ~ 2,
      Year == "3rd" ~ 3
    )
  )

df_species2 <- df_species %>%
  mutate(

```

```

    Species = factor(Species),
    Combination_ID = factor(Combination_ID),
    Component = factor(Component)
  )

fit_component_trend <- function(dat) {

  m <- lmer(
    Value ~ Year_Num + (1 | Species) + (1 | Combination_ID),
    data = dat,
    REML = TRUE
  )

  co <- summary(m)$coefficients["Year_Num", ]

  r2 <- r.squaredGLMM(m)[1] # marginal R2

  tibble(
    slope = unname(co["Estimate"]),
    se     = unname(co["Std. Error"]),
    p      = unname(co["Pr(>|t|)"]),
    star   = case_when(
      p < 0.001 ~ "***",
      p < 0.01  ~ "**",
      p < 0.05  ~ "*",
      p < 0.1   ~ ".",
      TRUE      ~ ""
    ),
    R2 = r2
  )
}

res_species_component <- df_species2 %>%
  group_by(Component, Component_Label) %>%
  group_modify(~ fit_component_trend(.x)) %>%
  ungroup()
res_species_component

my_colors <- c(
  "Above-ground" = "#A5B68D",
  "Below-ground" = "#AB886D",
  "Trait*Trait"  = "#FFE1AF",
  "Environment"  = "#7C93C3",
  "Trait*Env"    = "#C2D9FF"
)

level_order <- c("Above-ground", "Below-ground", "Trait*Trait", "Environment", "Trait*Env")
df_species$Component_Label <- factor(df_species$Component_Label, levels = level_order)
df_community$Component_Label <- factor(df_community$Component_Label, levels = level_order)

my_colors <- c(
  "Environment" = "#7C93C3",
  "Above-ground" = "#A5B68D",
  "Below-ground" = "#AB886D",
  "Trait*Trait"  = "#FFE1AF",
  "Trait*Env"    = "#C2D9FF"
)
slopes_species <- df_species %>%
  mutate(Year_Num = as.numeric(Year_Num)) %>%
  group_by(Component_Label) %>%
  do({
    m <- lm(Value ~ Year_Num, data = .)
    broom::tidy(m, conf.int = TRUE) %>%
      filter(term == "Year_Num")
  }) %>%
  ungroup() %>%
  transmute(
    Component_Label,
    slope = estimate,
    conf.low,
    conf.high,
    p.value)

comp_levels <- levels(df_species$Component_Label)

slopes_species <- slopes_species %>%
  mutate(
    Component_Label = factor(Component_Label, levels = comp_levels),
    p_lab = case_when(
      p.value < 0.001 ~ "P < 0.001",
      TRUE ~ paste0("P = ", formatC(p.value, format = "f", digits = 3))
    )
  )

```

```

    ),
    lab = paste0("slope = ", formatC(slope, format = "f", digits = 3),
                ", ", p_lab)

  )

y_rng <- diff(range(df_species$Value, na.rm = TRUE))

slopes_species <- slopes_species %>%
  arrange(Component_Label) %>%
  mutate(
    x_pos = 3.04,
    # Stagger the five labels in a clear band below the two dominant lines.
    y_pos = seq(max(df_species$Value, na.rm = TRUE) * 0.50,
                max(df_species$Value, na.rm = TRUE) * 0.16,
                length.out = n())
  )

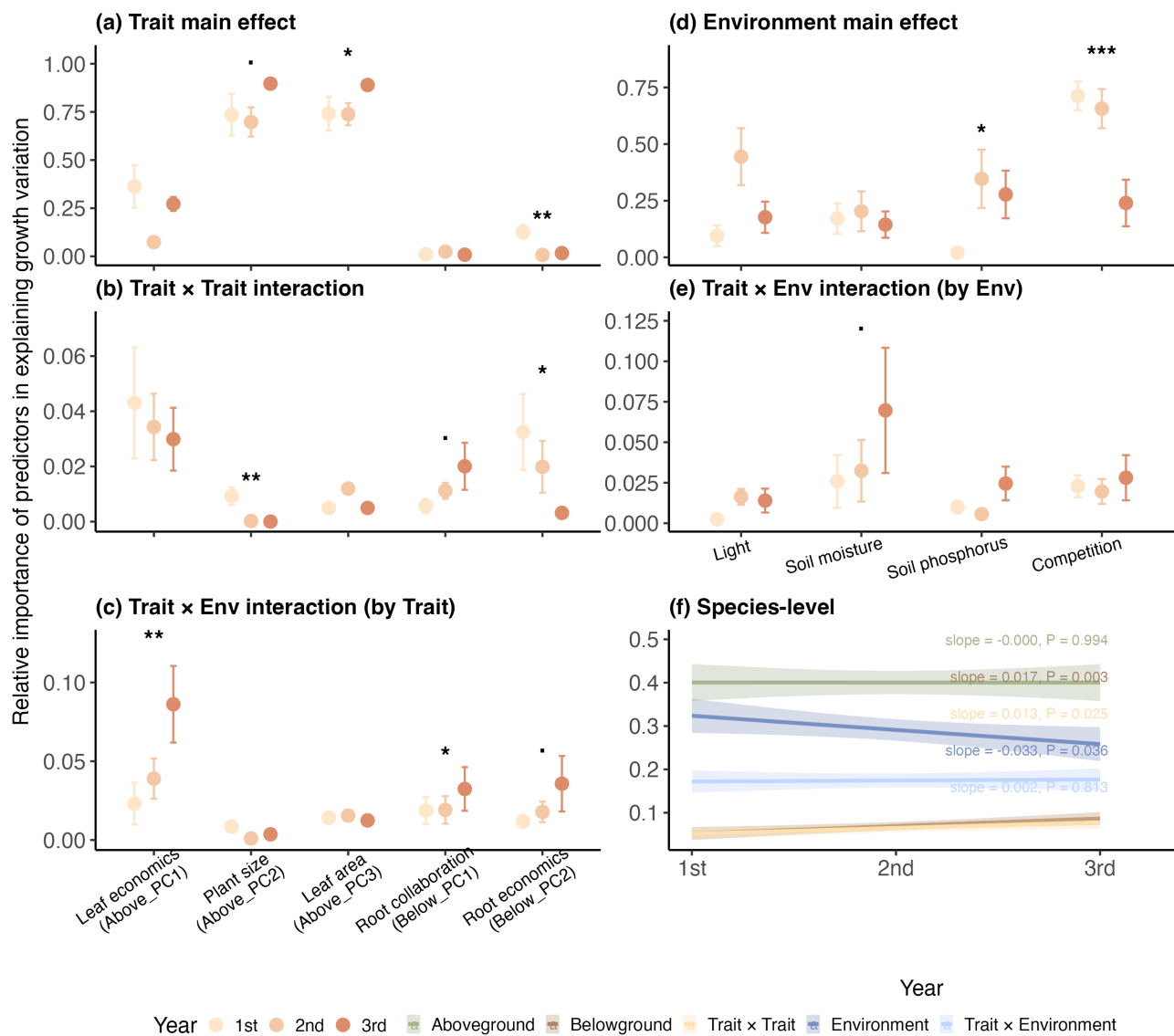
p_bs <- ggplot(df_species, aes(x = Year_Num, y = Value, color = Component_Label, fill = Component_Label)) +
  geom_smooth(aes(group = Component_Label), method = "lm", se = TRUE, linewidth = 1, alpha = 0.3) +
  scale_color_manual(values = my_colors, labels = component_legend_labels, name = NULL) +
  scale_fill_manual(values = my_colors, labels = component_legend_labels, name = NULL) +
  scale_x_continuous(breaks = 1:3, labels = c("1st", "2nd", "3rd")) +
  labs(title = "(f) Species-level", y = NULL, x = "Year") +
  theme_Publication() + panel_margin +
  theme(
    legend.position = "none",
    strip.background = element_blank(),
    strip.text = element_text(face = "bold", size = 11)
  ) +
  # Match slope-label colours to the fitted lines.
  geom_text(
    data = slopes_species,
    aes(x = x_pos, y = y_pos, label = lab, color = Component_Label),
    inherit.aes = FALSE,
    hjust = 1,
    size = 2.8
  ) +
  # Add space on the right to avoid clipping labels.
  coord_cartesian(xlim = c(1, 3.25), clip = "off")
fig4a_f <- p_a + p_d + p_b + p_e + p_c + p_bs + plot_layout(ncol=2)
fig4a_f <- fig4a_f + plot_layout(widths = c(1, 1), guides = "collect") & theme(legend.position = "bottom")

ggsave("outputs/figs/fig4_sp_var_imp_final.pdf", plot = fig4a_f, width = 7, height = 8)

fig4_a_panel <- p_a      # Figure 4a: trait main effects.
fig4_b_panel <- p_b      # Figure 4b: trait-trait interactions.

```

Figure 4. Temporal change in relative contributions



6. Robustness analysis

These models include all trait axes, environmental predictors and specified two-way interactions within each harvest year. They are used as a community-level sensitivity analysis because equivalent species-level global models are overparameterized and do not converge reliably.

These models provide a robustness analysis of the conclusions shown in Figures 3 and 4. They evaluate whether the temporal patterns remain when all prespecified trait axes, environmental predictors and interactions are fitted simultaneously within each harvest year.

One community-level global model was fitted for each harvest year.

```

# 1. Fit the year-specific global models
formula_full <- RGR_scale ~ Light + Moisture + AP + Compete + growth_time +
  Above_PC1 + Above_PC2 + Above_PC3 + Below_PC1 + Below_PC2 +
  Above_PC1:Below_PC1 + Above_PC2:Below_PC1 + Above_PC3:Below_PC1 +
  Above_PC1:Below_PC2 + Above_PC2:Below_PC2 + Above_PC3:Below_PC2 +
  Below_PC1:Below_PC2 + Above_PC1:Above_PC2 + Above_PC1:Above_PC3 +
  Above_PC2:Above_PC3 + Above_PC1:Light + Above_PC1:Moisture + Above_PC1:AP + Above_PC1:Compete +
  Above_PC2:Light + Above_PC2:Moisture + Above_PC2:AP + Above_PC2:Compete +
  Above_PC3:Light + Above_PC3:Moisture + Above_PC3:AP + Above_PC3:Compete +
  Below_PC1:Light + Below_PC1:Moisture + Below_PC1:AP + Below_PC1:Compete +
  Below_PC2:Light + Below_PC2:Moisture + Below_PC2:AP + Below_PC2:Compete +
  (1 | sp) + (1 | block/competition/pot_id)

# Fit the year-specific global models with the updated random-effects structure.
lm1 <- lmer(formula_full, data = tda_1st, REML = FALSE)
lm2 <- lmer(formula_full, data = tda_2nd, REML = FALSE)
lm3 <- lmer(formula_full, data = tda_3rd, REML = FALSE)

# Readable aliases; the original object names are retained for downstream code.
community_global_year1 <- lm1
community_global_year2 <- lm2
community_global_year3 <- lm3

# 2. Define the coefficient order, including competition
target_order <- c(
  # --- Single Predictors ---
  "Above_PC1", "Above_PC2", "Above_PC3",
  "Below_PC1", "Below_PC2",
  "Light", "Moisture", "AP", "Compete",

  # --- Trait x Trait ---
  "Above_PC1:Below_PC1", "Above_PC1:Below_PC2",
  "Above_PC2:Below_PC1", "Above_PC2:Below_PC2",
  "Above_PC3:Below_PC1", "Above_PC3:Below_PC2",
  "Below_PC1:Below_PC2",
  "Above_PC1:Above_PC2", "Above_PC1:Above_PC3", "Above_PC2:Above_PC3",

  # --- Trait x Env ---
  "Above_PC1:Light", "Above_PC1:Moisture", "Above_PC1:AP", "Above_PC1:Compete",
  "Above_PC2:Light", "Above_PC2:Moisture", "Above_PC2:AP", "Above_PC2:Compete",
  "Above_PC3:Light", "Above_PC3:Moisture", "Above_PC3:AP", "Above_PC3:Compete",
  "Below_PC1:Light", "Below_PC1:Moisture", "Below_PC1:AP", "Below_PC1:Compete",
  "Below_PC2:Light", "Below_PC2:Moisture", "Below_PC2:AP", "Below_PC2:Compete"
)

# 3. Match coefficient names across model outputs
get_model_data <- function(model, target_vars) {
  summ <- summary(model)
  coefs <- as.data.frame(summ$coefficients)
  coefs$RawName <- rownames(coefs)
  matched_data <- list()

  for (target in target_vars) {
    found <- FALSE
    # First try an exact match.
    if (target %in% coefs$RawName) {
      row <- coefs[coefs$RawName == target, ]
      row$DisplayName <- target
      matched_data[[target]] <- row
      found <- TRUE
    }
    # Then try the reversed interaction order.
    if (!found && grepl(":", target)) {
      parts <- str_split(target, ":")[[1]]
      reversed <- paste(parts[2], parts[1], sep = ":")
      if (reversed %in% coefs$RawName) {
        row <- coefs[coefs$RawName == reversed, ]
        row$DisplayName <- target
        matched_data[[target]] <- row
        found <- TRUE
      }
    }
    # Finally match the factor-coded competition term.
    if (!found && grepl("Compete", target)) {
      target_pattern <- gsub("Compete", "Compete.*", target)
      match_idx <- grep(paste0("^", target_pattern, "$"), coefs$RawName)
      if (length(match_idx) == 0 && grepl(":", target)) {
        parts <- str_split(target, ":")[[1]]
        rev_pattern <- paste0(gsub("Compete", "Compete.*", parts[2]), ":", gsub("Compete", "Compete.*", parts[1]))
        match_idx <- grep(paste0("^", rev_pattern, "$"), coefs$RawName)
      }
    }
  }
}

```

```

    if (length(match_idx) > 0) {
      actual_name <- coefs$RawName[match_idx[1]]
      row <- coefs[coefs$RawName == actual_name, ]
      row$DisplayName <- target
      matched_data[[target]] <- row
      found <- TRUE
    }
  }
}
final_df <- do.call(rbind, matched_data)
if (is.null(final_df)) return(data.frame())

final_df <- final_df %>%
  mutate(
    Variable = factor(DisplayName, levels = rev(target_vars)),
    Estimate = Estimate,
    Lower = Estimate - 1.96 * `Std. Error`,
    Upper = Estimate + 1.96 * `Std. Error`,
    P_val = `Pr(>|t|)`
  )
return(final_df)
}

# 4. Construct the robustness forest plots
create_forest_plot <- function(model, title_text, target_vars) {

  # Calculate marginal R-squared.
  r2_vals <- MuMIn::r.squaredGLMM(model)
  # Extract the variance explained by fixed effects.
  # r.squaredGLMM returns marginal and conditional R-squared.
  r2_m <- round(r2_vals[1, "R2m"], 3)
  r2_c <- round(r2_vals[1, "R2c"], 3)

  # Construct the subtitle.
  subtitle_text <- bquote(italic(R)[m]^2 == .(r2_m))

  # Extract coefficients.
  plot_data <- get_model_data(model, target_vars)

  if (nrow(plot_data) == 0) return(ggplot() + labs(title = "No Data"))

  plot_data$col <- case_when(
    plot_data$P_val < 0.05 ~ "black",
    plot_data$P_val < 0.10 ~ "grey70",
    TRUE ~ "white"
  )

  p <- ggplot(plot_data, aes(x = Estimate, y = Variable)) +
    scale_fill_identity() +
    geom_vline(xintercept = 0, linetype = "dashed", color = "grey60") +
    geom_errorbarh(aes(xmin = Lower, xmax = Upper), height = 0.2, color = "black") +
    geom_point(size = 2.5, shape = 21, aes(fill = col), color = "black") +
    scale_fill_identity() +
    scale_y_discrete(position = "left") +

    # Add marginal R-squared as the subtitle.
    labs(x = "Standardized Estimate", y = NULL,
         title = title_text,
         subtitle = subtitle_text) +

    theme_classic() +
    theme(
      axis.text.y = element_text(size = 10.5, color = "black"),
      plot.title = element_text(hjust = 0.5, size = 11, face = "bold"),
      plot.subtitle = element_text(hjust = 0.5, size = 9, color = "grey30"), # Subtitle style.
      plot.margin = margin(5, 5, 5, 5)
    )

  return(p)
}

p1 <- create_forest_plot(lm1, "(a) 1st Year", target_order) + labs(x=NULL)
p2 <- create_forest_plot(lm2, "(b) 2nd Year", target_order)
p3 <- create_forest_plot(lm3, "(c) 3rd Year", target_order) + labs(x=NULL)

p2 <- p2 + theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())
p3 <- p3 + theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())

final_forest <- p1 + p2 + p3 + plot_layout(ncol = 3, widths = c(1, 1, 1))

supplementary_figure_dir <- file.path(output_root, "supplementary_figures")
dir.create(supplementary_figure_dir, recursive = TRUE, showWarnings = FALSE)

```

```
ggsave(file.path(supplementary_figure_dir, "Figure_S8_year_specific_global_models.pdf"),  
       plot = final_forest, width = 8, height = 8, device = cairo_pdf, bg = "white")
```

The corresponding forest plot is presented as Figure S8 in the supplementary-figure section.

7. Supplementary effect sizes

The pooled temporal community-level global model contains all trait axes, environmental variables, specified trait-trait and trait-environment interactions, and their interactions with seedling age. This supplementary analysis describes the direction and magnitude of individual effects and how those effect sizes change with seedling age. It does not replace the marginal-R-squared tests of H1-H3.

```
total_scaled_tda1 <- pcatraits_tda1
lme_full <- lmerTest::lmer(
  RGR ~ Above_PC1 + Above_PC2 + Above_PC3 + Below_PC1 + Below_PC2 +
    Light + Moisture + AP + Compete + growth_time +
    Above_PC1:growth_time + Above_PC2:growth_time + Above_PC3:growth_time +
    Below_PC1:growth_time + Below_PC2:growth_time +
    Light:growth_time + Moisture:growth_time + AP:growth_time + Compete:growth_time +
    Above_PC1:Below_PC1 + Above_PC2:Below_PC1 + Above_PC3:Below_PC1 +
    Above_PC1:Below_PC2 + Above_PC2:Below_PC2 + Above_PC3:Below_PC2 +
    Below_PC1:Below_PC2 + Above_PC1:Above_PC2 + Above_PC1:Above_PC3 + Above_PC2:Above_PC3 +
    Above_PC1:Light + Above_PC1:Moisture + Above_PC1:AP + Above_PC1:Compete +
    Above_PC2:Light + Above_PC2:Moisture + Above_PC2:AP + Above_PC2:Compete +
    Above_PC3:Light + Above_PC3:Moisture + Above_PC3:AP + Above_PC3:Compete +
    Below_PC1:Light + Below_PC1:Moisture + Below_PC1:AP + Below_PC1:Compete +
    Below_PC2:Light + Below_PC2:Moisture + Below_PC2:AP + Below_PC2:Compete +
    Above_PC1:Below_PC1:growth_time + Above_PC2:Below_PC1:growth_time + Above_PC3:Below_PC1:growth_time +
    Above_PC1:Below_PC2:growth_time + Above_PC2:Below_PC2:growth_time + Above_PC3:Below_PC2:growth_time +
    Below_PC1:Below_PC2:growth_time + Above_PC1:Above_PC2:growth_time + Above_PC1:Above_PC3:growth_time +
    Above_PC2:Above_PC3:growth_time +
    Above_PC1:Light:growth_time + Above_PC1:Moisture:growth_time + Above_PC1:AP:growth_time +
    Above_PC1:Compete:growth_time +
    Above_PC2:Light:growth_time + Above_PC2:Moisture:growth_time + Above_PC2:AP:growth_time +
    Above_PC2:Compete:growth_time +
    Above_PC3:Light:growth_time + Above_PC3:Moisture:growth_time + Above_PC3:AP:growth_time +
    Above_PC3:Compete:growth_time +
    Below_PC1:Light:growth_time + Below_PC1:Moisture:growth_time + Below_PC1:AP:growth_time +
    Below_PC1:Compete:growth_time +
    Below_PC2:Light:growth_time + Below_PC2:Moisture:growth_time + Below_PC2:AP:growth_time +
    Below_PC2:Compete:growth_time +
    (1 | sp) + (1 | block/competition/pot_id),
  data = total_scaled_tda1
)

# Readable alias retained alongside the original object name.
pooled_temporal_global_model <- lme_full
```

```
## =====
## 1) Forest plot: robust term-finding (handles factor like Compete)
## =====

forest_var_paired_sig_offset2 <- function(model,
                                           vars,
                                           var_labels,
                                           time_var = "growth_time",
                                           title = "(a)",
                                           xlab = "Effect size",
                                           y_offset = 0.15) {

  cf <- fixef(model)
  cn <- names(cf)
  ci <- confint(model, method = "Wald")
  sm <- summary(model)$coefficients

  # helper: swap A:B <-> B:A
  swap_term <- function(x) {
    if (!grepl(":", x)) return(x)
    paste(rev(strsplit(x, ":")[1]), collapse = ":")
  }

  # helper: find exact or swapped; if not found, allow prefix match for factor terms
  find_term_robust <- function(var, type = c("Main", "Time")) {
    type <- match.arg(type)

    # target interaction term string (ideal)
    term0 <- if (type == "Main") var else paste0(var, ":", time_var)

    # 1) exact
    if (term0 %in% cn) return(term0)
    # 2) swapped
    term1 <- swap_term(term0)
    if (term1 %in% cn) return(term1)

    # 3) prefix match (for factor-coded variables, e.g., Competeinter)
    # Main: grep("^Compete", cn) but exclude ":" interactions
    # Time: grep("^Compete.*:growth_time$" or "^growth_time:Compete", etc.)
    if (type == "Main") {
      hits <- grep(paste0("^", var), cn, value = TRUE)
      hits <- hits[!grepl(":", hits)]
      # remove intercept if someone passes "(Intercept)" by mistake
      hits <- hits[hits != "(Intercept)"]
      if (length(hits) == 1) return(hits)
      if (length(hits) > 1) {
        message("Multiple main-effect terms matched for ", var, ":",
                paste(hits, collapse = ", "),
                ". Using the first: ", hits[1])
        return(hits[1])
      }
    } else {
      # allow both var*:growth_time and growth_time:var*
      hits <- grep(paste0("^", var, ".*:", time_var, "$"), cn, value = TRUE)
      if (length(hits) == 0) {
        hits <- grep(paste0("^", time_var, ":", var), cn, value = TRUE)
      }
      if (length(hits) == 1) return(hits)
      if (length(hits) > 1) {
        message("Multiple time-interaction terms matched for ", var, ":",
                paste(hits, collapse = ", "),
                ". Using the first: ", hits[1])
        return(hits[1])
      }
    }
  }

  stop("Term not found for variable '", var, "' (", type, ").\n",
       "Available coefficient names include:\n",
       paste(head(cn, 40), collapse = ", "),
       if (length(cn) > 40) "\n... (truncated)" else "")
}

# build y-axis table
var_df <- tibble(
  Var = vars,
  label = var_labels,
  y_base = rev(seq_along(vars))
)

# extract one row per (Var, Effect)
get_effect <- function(v, type) {
```

```

term <- find_term_robust(v, type)
tibble(
  Var = v,
  Effect = type,
  term = term,
  estimate = unname(cf[term]),
  lower = ci[term, 1],
  upper = ci[term, 2],
  p = sm[term, "Pr(>|t|)"]
)
}

df <- bind_rows(
  lapply(vars, get_effect, type = "Main"),
  lapply(vars, get_effect, type = "Time")
) %>%
left_join(var_df, by = "Var") %>%
mutate(
  y = ifelse(Effect == "Main", y_base + y_offset, y_base - y_offset),
  Sig = case_when(
    p < 0.05 ~ "p < 0.05",
    p < 0.10 ~ "0.05-0.10",
    TRUE ~ "p ≥ 0.10"
  ),
  Effect = factor(Effect, levels = c("Main", "Time"))
)

ggplot(df, aes(x = estimate, y = y)) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  geom_errorbarh(aes(xmin = lower, xmax = upper, colour = Effect),
    height = 0.12, linewidth = 0.6) +
  geom_point(aes(fill = Sig, colour = Effect),
    shape = 21, size = 2.2, stroke = 0.9) +
  scale_fill_manual(
    values = c("p < 0.05" = "black", "0.05-0.10" = "grey70", "p ≥ 0.10" = "white"),
    name = "P value",
    guide = "none"
  ) +
  scale_color_manual(values = c("Main" = "black", "Time" = "#F3A26D"),
    labels = c("main effects of predictor", "predictor × year interaction effects"),
    name = NULL) +
  scale_y_continuous(breaks = var_df$y_base, labels = var_df$label) +
  labs(title = title, x = xlab, y = NULL) +
  theme_Publication() +
  theme(
    panel.background = element_rect(fill = "white", colour = NA),
    plot.background = element_rect(fill = "white", colour = NA),

    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank()
  )
}

vars9 <- c("Above_PC1", "Above_PC2", "Above_PC3", "Below_PC1", "Below_PC2",
  "Light", "Moisture", "AP", "Compete")

labels9 <- c("Leaf economics\n(Above_PC1)",
  "Plant size\n(Above_PC2)",
  "Leaf area\n(Above_PC3)",
  "Root collaboration\n(Below_PC1)",
  "Root economics\n(Below_PC2)",
  "Light intensity",
  "Soil moisture",
  "Soil phosphorus",
  "Competition")

forest9 <- forest_var_paired_sig_offset2(
  model = lme_full,
  vars = vars9,
  var_labels = labels9,
  title = "(a)"
)

```

```

year_cols <- c("#F5E7C6", "#F7B980", "#DE802B")
plot_trait_year_raw <- function(model_raw,
                                data,
                                trait,
                                time_var = "growth_time",
                                response_raw = "RGR",
                                time_vals,
                                time_labels = c("1st", "2nd", "3rd"),
                                trait_range = NULL,
                                title = NULL,
                                colors = year_cols,
                                show_ci = TRUE) {

  pred <- ggpredict(
    model_raw,
    terms = c(
      trait,
      sprintf("%s [%s]", time_var, paste(time_vals, collapse = ", "))
    ),
    type = "fixed"
  )

  pred_df <- as.data.frame(pred) %>%
    mutate(
      Year = factor(group,
                    levels = as.character(time_vals),
                    labels = time_labels)
    )

  pal <- setNames(colors, time_labels)

  p <- ggplot() +
    geom_point(
      data = data,
      aes(x = .data[[trait]], y = .data[[response_raw]], colour = comp_year),
      alpha = 0.1,
      size = 0.2
    ) +
    scale_color_manual(values = colors)

  if (show_ci) {
    p <- p + geom_ribbon(
      data = pred_df,
      aes(x = x, ymin = conf.low, ymax = conf.high, fill = Year),
      alpha = 0.15,
      colour = NA
    )
  }

  p <- p +
    geom_line(
      data = pred_df,
      aes(x = x, y = predicted, colour = Year),
      linewidth = 1.2
    ) +
    scale_color_manual(values = pal, name = "Year") +
    scale_fill_manual(values = pal, name = "Year") +
    guides(fill = "none") +
    labs(x = trait, y = "RGR", title = title) +
    theme_Publication(base_size=11)+
    theme(
      axis.text.x = element_text(size = 10),
      axis.text.y = element_text(size = 10)) +
    scale_y_continuous(
      labels = number_format(accuracy = 0.1)
    ) +
    scale_x_continuous(
      breaks = function(x) floor(min(x)):ceiling(max(x))
    )

  if (!is.null(trait_range)) {
    p <- p + coord_cartesian(xlim = trait_range, ylim = c(0, 1.2))
  } else {
    p <- p + coord_cartesian(ylim = c(0, NA))
  }

  return(p)
}

gt_vals <- sort(as.numeric(unique(total_scaled_tda1$growth_time)))

```

```

lme_plot_raw=lme_full

trait_vars <- c(
  "Above_PC1",
  "Above_PC2",
  "Above_PC3",
  "Below_PC1",
  "Below_PC2")

trait_ranges <- lapply(
  total_scaled_tda1[trait_vars],
  function(x) quantile(x, probs = c(0.001,0.999), na.rm = TRUE))

above_pc1_p <- plot_trait_year_raw(
  model_raw = lme_plot_raw,
  data = total_scaled_tda1,
  trait = "Above_PC1",
  time_vals = gt_vals,
  trait_range = trait_ranges$Above_PC1,
  title = "(b)" + xlab("Leaf economics")+ylab(NULL)+
  scale_x_continuous(breaks = c(-4,-2, 0, 2,4) )+
  theme(plot.margin = margin(t = 1, r = 1, b = 1, l = 1, unit = "pt"))

above_pc2_p <- plot_trait_year_raw(lme_plot_raw, total_scaled_tda1, "Above_PC2",
  time_vals = gt_vals,
  trait_range = trait_ranges$Above_PC2,
  title="(c)" + ylab(NULL) + xlab("Plant size") +
  scale_x_continuous(breaks = c(-4,-2, 0, 2,4) )+
  theme(plot.margin = margin(t = 1, r = 1, b = 1, l = 1, unit = "pt"))

above_pc3_p <- plot_trait_year_raw(lme_plot_raw, total_scaled_tda1, "Above_PC3",
  time_vals = gt_vals,
  trait_range = trait_ranges$Above_PC3,
  title="(d)" + ylab(NULL) + xlab("Leaf area")+
  scale_x_continuous(breaks = c(-4,-2, 0, 2,4) )+
  theme(plot.margin = margin(t = 1, r = 1, b = 1, l = 1, unit = "pt"))

below_pc1_p <- plot_trait_year_raw(lme_plot_raw, total_scaled_tda1, "Below_PC1",
  time_vals = gt_vals,
  trait_range = trait_ranges$Below_PC1,
  title="(e)" + xlab("Root collaboration") +
  scale_x_continuous(breaks = c(-4,-2, 0, 2,4) ) +
  ylab(expression("Relative growth rate (RGR, log cm yr"^-1)"))+
  theme(plot.margin = margin(t = 1, r = 1, b = 1, l = 1, unit = "pt"))

below_pc2_p <- plot_trait_year_raw(lme_plot_raw, total_scaled_tda1, "Below_PC2",
  time_vals = gt_vals,
  trait_range = trait_ranges$Below_PC2,
  title="(f)" + ylab(NULL) + xlab("Root economics") +
  scale_x_continuous(breaks = c(-4,-2, 0, 2,4) )+
  theme(plot.margin = margin(t = 1, r = 1, b = 1, l = 1, unit = "pt"))

plot_env_year <- function(model,
  data,
  env,
  response = "RGR",
  time_var = "growth_time",
  time_vals,
  time_labels = c("1st", "2nd", "3rd"),
  env_range = NULL,
  title = NULL,
  colors = year_cols,
  show_points = TRUE,
  point_alpha = 0.20,
  point_size = 0.6,
  ci_alpha = 0.12) {

  pred <- ggpredict(
    model,
    terms = c(
      env,
      sprintf("%s [%s]", time_var, paste(time_vals, collapse = ", "))
    ),
    type = "fixed"
  )
}

```

```

pred_df <- as.data.frame(pred) %>%
  mutate(
    Year = factor(group,
                  levels = as.character(time_vals),
                  labels = time_labels)
  )

pal <- setNames(colors, time_labels)

p <- ggplot()
if (show_points) {
  p <- p +
    geom_point(
      data = data,
      aes(x = .data[[env]], y = .data[[response]], colour = comp_year),
      alpha = 0.1,
      size = 0.2
    ) + scale_color_manual(values = colors)
}

p <- p +
  geom_ribbon(
    data = pred_df,
    aes(x = x, ymin = conf.low, ymax = conf.high, fill = Year),
    alpha = ci_alpha,
    colour = NA
  ) +
  geom_line(
    data = pred_df,
    aes(x = x, y = predicted, colour = Year),
    linewidth = 1.2
  ) +
  scale_color_manual(values = pal, name = "Year") +
  scale_fill_manual(values = pal, name = "Year") +
  guides(fill = "none") +
  labs(x = env, y = "RGR", title = title) +
  theme_Publication(base_size = 11) +
  theme(
    axis.text.x = element_text(size = 10),
    axis.text.y = element_text(size = 10)) +
  scale_y_continuous(
    labels = number_format(accuracy = 0.1))
# Restrict predictions to the observed environmental range and non-negative RGR.
if (!is.null(env_range)) {
  p <- p + coord_cartesian(xlim = env_range, ylim = c(0, NA))
} else {
  p <- p + coord_cartesian(ylim = c(0, NA))
}

return(p)
}

lme_env_raw = lme_full
env_ranges <- list(
  Light = quantile(total_scaled_tda1$Light, c(0.05, 0.95), na.rm = TRUE),
  Moisture = quantile(total_scaled_tda1$Moisture, c(0.05, 0.95), na.rm = TRUE),
  AP = quantile(total_scaled_tda1$AP, c(0.05, 0.95), na.rm = TRUE)
)

p_light <- plot_env_year(
  model = lme_env_raw,
  data = total_scaled_tda1,
  env = "Light",
  time_vals = gt_vals,
  env_range = env_ranges$Light,
  title = "(g)" + ylab(NULL) + scale_x_continuous(breaks = c(-1, 0, 1)) + theme(plot.margin = margin(t = 1, r = 1, b = 1, l = 1, unit = "pt")) + xlab("Light intensity")
)

p_moist <- plot_env_year(
  lme_env_raw, total_scaled_tda1,
  env = "Moisture",
  time_vals = gt_vals,
  env_range = env_ranges$Moisture,
  title = "(h)"
) + theme(plot.margin = margin(t = 1, r = 1, b = 1, l = 1, unit = "pt")) + ylab(NULL) + xlab("Soil moisture")

p_ap <- plot_env_year(
  lme_env_raw, total_scaled_tda1,
  env = "AP",
  time_vals = gt_vals,
  env_range = env_ranges$AP,
  title = "(i)"
)

```

```

)+ylab(NULL))+theme(plot.margin = margin(t = 1, r = 1, b = 1, l = 1, unit = "pt"))+xlab("Soil phosphorus")

p_compete <- plot_env_year(
  lme_env_raw, total_scaled_tda1,
  env = "Compete",
  time_vals = gt_vals,
  title = "(j)",
  show_points = TRUE
)+ylab(NULL))+theme(plot.margin = margin(t = 1, r = 1, b = 1, l = 1, unit = "pt"))+xlab("Competition")

fig5a <- forest9 + theme_Publication(base_size = 11) +
  theme(
    axis.text.x = element_text(size = 10),
    axis.text.y = element_text(size = 11))

## ---- Left column: (a) only uses 70% height ----
fig5a_short <- (
  fig5a +
  theme(
    legend.position = "none"
  )
)

## ---- Right block: 3x3 panels (unchanged) ----
p_right <- (above_pc1_p + above_pc2_p + above_pc3_p +
  below_pc1_p + below_pc2_p + p_light +
  p_moist + p_ap + p_compete) +
  plot_layout(ncol = 3) &
  theme(legend.position = "bottom")

## ---- Combine left + right with width ratio 1.2 : 3 ----
fig5_all <- (fig5a_short | p_right) +
  plot_layout(widths = c(1, 3), guides = "collect") &
  theme(
    legend.position = "bottom",
    legend.text = element_text(size = 10),
    legend.title = element_text(size = 10)
  )

# Export at double-column width (7 in); text sizes are reduced accordingly.
fig5_export <- fig5_all &
  theme(
    plot.title = element_text(size = 10, face = "bold"),
    axis.title = element_text(size = 9),
    axis.text.x = element_text(size = 7.5),
    axis.text.y = element_text(size = 7.5),
    legend.text = element_text(size = 8),
    legend.title = element_text(size = 8),
    legend.key.size = unit(0.35, "cm"),
    legend.margin = margin(0, 0, 0, 0)
  )

ggsave(fig5_export,
  file = "outputs/figs/fig5_trait_effect.pdf",
  height = 5.2, width = 7, bg = "white")

```

Figure 5. Temporal change in main effects

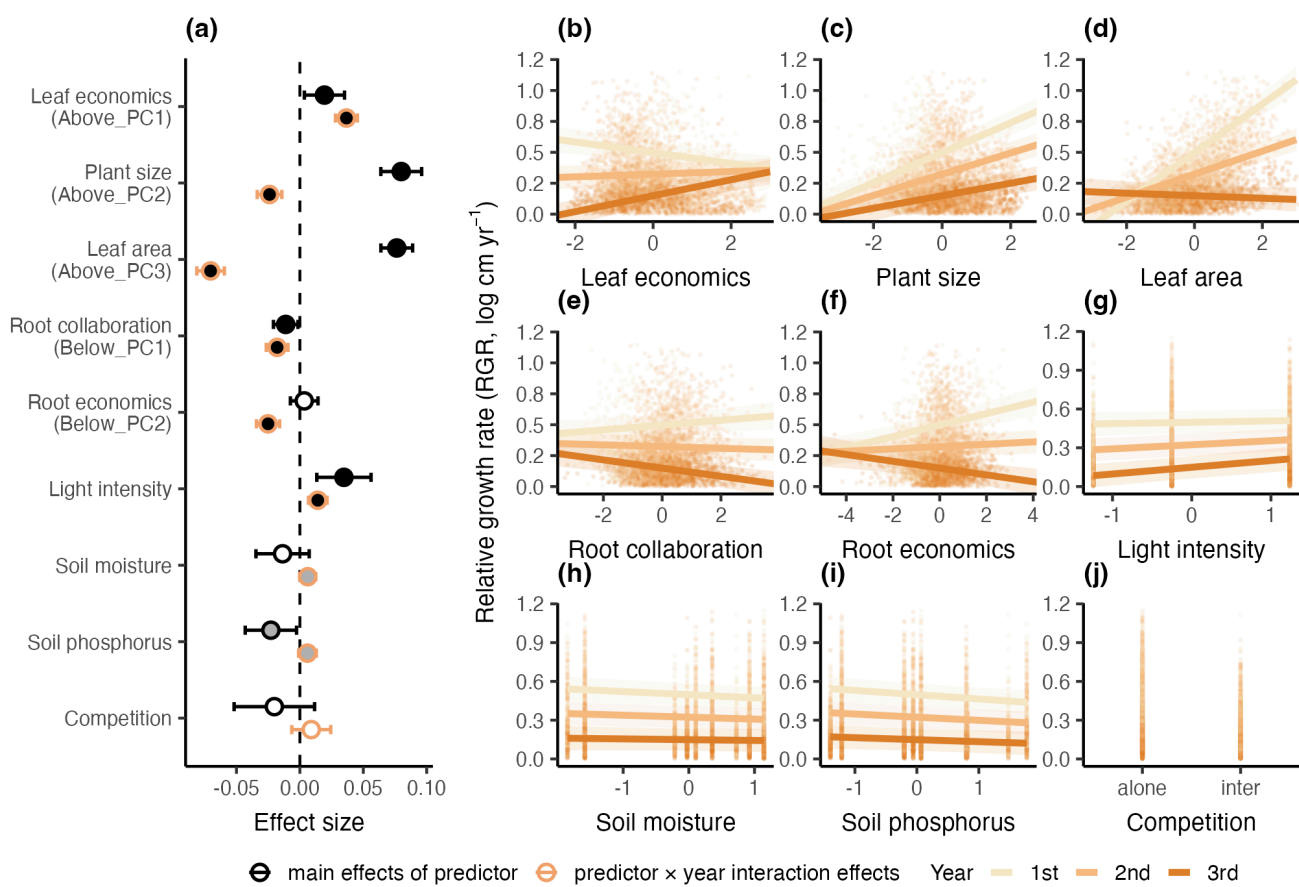


Figure 6 combines standardized interaction coefficients from the pooled full model with three representative three-way interaction predictions. Forest plots retain interactions for which either the interaction main effect or its interaction with year has $P < 0.10$.

```

figure6_model <- lme_full
figure6_data <- total_scaled_tda1
figure6_years <- c("1st", "2nd", "3rd")
figure6_time_values <- sort(unique(figure6_data$growth_time))
figure6_colours <- c("#B7B2D0", "#E5A15A", "#2F8064")

figure6_theme <- function(base_size = 11) {
  theme_classic(base_size = base_size, base_family = "sans") +
    theme(
      plot.title = element_text(face = "bold", hjust = 0, size = base_size + 1),
      axis.title = element_text(colour = "black"),
      axis.text = element_text(colour = "grey25"),
      legend.position = "bottom",
      strip.background = element_blank(),
      strip.text = element_text(face = "bold"),
      plot.background = element_rect(fill = "white", colour = NA),
      panel.background = element_rect(fill = "white", colour = NA),
      plot.margin = margin(5, 6, 5, 6)
    )
}

figure6_coefficients <- broom.mixed::tidy(
  figure6_model,
  effects = "fixed",
  conf.int = TRUE,
  conf.method = "Wald"
)

find_figure6_coefficient <- function(coefficient_table, components) {
  candidate <- vapply(
    strsplit(coefficient_table$term, ":", fixed = TRUE),
    function(x) identical(sort(x), sort(components)),
    logical(1)
  )
  if (sum(candidate) != 1) {
    stop("Expected one coefficient for: ", paste(components, collapse = " x "))
  }
  coefficient_table[candidate, ]
}

trait_axes_figure6 <- c("Above_PC1", "Above_PC2", "Above_PC3", "Below_PC1", "Below_PC2")
environment_terms_figure6 <- c("Light", "Moisture", "AP", "Competeinter")

all_tt_figure6 <- combn(trait_axes_figure6, 2, simplify = FALSE)
all_te_figure6 <- unlist(
  lapply(trait_axes_figure6, function(trait) {
    lapply(environment_terms_figure6, function(environment) c(trait, environment))
  }),
  recursive = FALSE
)

short_names_figure6 <- c(
  "Above_PC1" = "Leaf econ", "Above_PC2" = "Plant size",
  "Above_PC3" = "Leaf area", "Below_PC1" = "Root collab",
  "Below_PC2" = "Root econ", "Light" = "Light",
  "Moisture" = "Moisture", "AP" = "AP",
  "Competeinter" = "Competition"
)

interaction_forest_data <- function(term_list) {
  output <- map_dfr(term_list, function(components) {
    interaction_id <- paste(components, collapse = "__")
    interaction_label <- paste(unname(short_names_figure6[components]), collapse = " x ")

    bind_rows(
      find_figure6_coefficient(figure6_coefficients, components) %>%
        mutate(Effect = "Interaction main effect"),
      find_figure6_coefficient(
        figure6_coefficients,
        c(components, "growth_time")
      ) %>%
        mutate(Effect = "Interaction x year effect")
    ) %>%
      mutate(Interaction = interaction_id, Label = interaction_label)
  }) %>%
  group_by(Interaction) %>%
  filter(any(p.value < 0.10, na.rm = TRUE)) %>%
  ungroup()

interaction_order <- unique(output$Interaction)
output %>%

```

```

mutate(
  Interaction = factor(Interaction, levels = rev(interaction_order)),
  Effect = factor(
    Effect,
    levels = c("Interaction main effect", "Interaction × year effect")
  ),
  Significance = case_when(
    p.value < 0.05 ~ "P < 0.05",
    p.value < 0.10 ~ "0.05 ≤ P < 0.10",
    TRUE ~ "P ≥ 0.10"
  ),
  y = as.numeric(Interaction) +
    if_else(Effect == "Interaction main effect", 0.12, -0.12)
)
}

plot_interaction_forest <- function(data, title, show_x = TRUE) {
  label_map <- data %>%
    distinct(Interaction, Label) %>%
    arrange(as.numeric(Interaction))

  ggplot(data, aes(x = estimate, y = y)) +
    geom_vline(xintercept = 0, linetype = "dashed") +
    geom_errorbarh(
      aes(xmin = conf.low, xmax = conf.high, colour = Effect),
      height = 0.08, linewidth = 0.6
    ) +
    geom_point(
      aes(fill = Significance, colour = Effect),
      shape = 21, size = 2.2, stroke = 0.85
    ) +
    scale_y_continuous(
      breaks = seq_len(nrow(label_map)),
      labels = label_map$Label
    ) +
    scale_colour_manual(values = c(
      "Interaction main effect" = "black",
      "Interaction × year effect" = "#F09A61"
    ), labels = c(
      "Interaction main effect" = "main effects of predictor",
      "Interaction × year effect" = "predictor × year interaction effects"
    )) +
    scale_fill_manual(values = c(
      "P < 0.05" = "black",
      "0.05 ≤ P < 0.10" = "grey70",
      "P ≥ 0.10" = "white"
    )) +
    labs(
      title = title,
      x = if (show_x) "Effect size" else NULL,
      y = NULL,
      colour = NULL,
      fill = "P value"
    ) +
    figure6_theme(8) +
    theme(
      plot.title = element_text(size = 9, face = "bold"),
      plot.title.position = "plot",
      axis.text.y = element_text(size = 7, colour = "black"),
      legend.position = "none"
    )
}

tt_forest_data <- interaction_forest_data(all_tt_figure6)
te_forest_data <- interaction_forest_data(all_te_figure6)

p6a <- plot_interaction_forest(
  tt_forest_data,
  "(a) Trait–trait interactions",
  show_x = FALSE
)
p6b <- plot_interaction_forest(
  te_forest_data,
  "(b) Trait–environment interactions"
)

plot_three_way <- function(x, moderator, title, x_label, moderator_label,
  y_limits = c(0, 0.75)) {
  moderator_values <- c(-1.5, 0, 1.5)
  prediction <- as.data.frame(ggeffects::ggpredict(
    figure6_model,
    terms = c(

```

```

    x,
    sprintf("%s [%s]", moderator, paste(moderator_values, collapse = ",")),
    sprintf(
      "growth_time [%s]",
      paste(figure6_time_values, collapse = ",")
    )
  ),
  type = "fixed"
)) %>%
mutate(
  Moderator = factor(
    as.character(group),
    levels = as.character(moderator_values),
    labels = c("Low", "Medium", "High")
  ),
  Year = factor(
    as.character(facet),
    levels = as.character(figure6_time_values),
    labels = figure6_years
  )
)

observed <- figure6_data %>%
  mutate(Year = factor(comp_year, levels = figure6_years))

# Original Figure 6 displays the central 5–95% of each focal predictor.
x_limits <- as.numeric(quantile(
  figure6_data[[x]], probs = c(0.05, 0.95), na.rm = TRUE
))

ggplot() +
  geom_point(
    data = observed,
    aes(x = .data[[x]], y = RGR),
    colour = "grey65", alpha = 0.14, size = 0.45
  ) +
  geom_line(
    data = prediction,
    aes(x = x, y = predicted, colour = Moderator),
    linewidth = 1.0
  ) +
  facet_wrap(
    ~ Year,
    nrow = 1,
    labeller = as_labeller(c(
      "1st" = "1st year", "2nd" = "2nd year", "3rd" = "3rd year"
    ))
  ) +
  scale_colour_manual(values = figure6_colours) +
  labs(title = title, x = x_label, y = NULL,
    colour = paste0(toupper(substr(moderator_label, 1, 1)), substring(moderator_label, 2, ":")) +
    coord_cartesian(xlim = x_limits, ylim = y_limits) +
    figure6_theme(8.5) +
    theme(
      plot.title = element_text(size = 8.5, face = "bold", margin = margin(b = 0)),
      axis.title = element_text(size = 8),
      axis.text = element_text(size = 7),
      strip.text = element_text(size = 7.5, face = "bold"),
      legend.position = "top",
      legend.justification = "right",
      legend.location = "plot",
      legend.direction = "horizontal",
      legend.title = element_text(size = 7.5, face = "bold"),
      legend.title.position = "left",
      legend.box.spacing = unit(1, "pt"),
      legend.background = element_blank(),
      legend.text = element_text(size = 7.5),
      legend.key.height = unit(0.22, "cm"),
      legend.key.width = unit(0.35, "cm"),
      legend.spacing.y = unit(0, "cm"),
      legend.margin = margin(0, 0, 0, 0)
    )
  )
}

p6c <- plot_three_way(
  "Above_PC1", "Below_PC2",
  "(c) Leaf economics × Root economics",
  "Leaf economics (Above_PC1)", "root economics",
  y_limits = c(0, 0.70)
)

p6d <- plot_three_way(
  "Below_PC1", "Light",

```

```

"(d) Root collaboration × Light",
"Root collaboration (Below_PC1)", "light",
y_limits = c(0, 0.70)
)
p6e <- plot_three_way(
  "Below_PC2", "Moisture",
  "(e) Root economics × Soil moisture",
  "Root economics (Below_PC2)", "moisture",
  y_limits = c(0, 0.75)
)

left_figure_6 <- p6a / p6b + plot_layout(heights = c(0.82, 1.5))
right_figure_6_grid <- p6c / p6d / p6e
right_figure_6 <- cowplot::ggdraw() +
  cowplot::draw_plot(
    right_figure_6_grid,
    x = 0.07, y = 0, width = 0.93, height = 1
  ) +
  cowplot::draw_label(
    "Relative growth rate (RGR, log cm yr-1)",
    x = 0.010, y = 0.5, angle = 90, size = 8
  )

# Both columns are placed at full height so that panel (c) aligns with (a) at
# the top and panel (e) with (b) at the bottom.
figure_6_body <- cowplot::plot_grid(
  left_figure_6, right_figure_6,
  nrow = 1, rel_widths = c(0.41, 0.59), align = "none"
)

interaction_legend <- cowplot::get_legend(
  p6a +
  guides(
    colour = guide_legend(order = 1, nrow = 1),
    fill = "none"
  ) +
  theme(
    legend.position = "bottom", legend.box = "horizontal",
    legend.text = element_text(size = 8.5),
    legend.key.width = unit(0.6, "cm")
  )
)

figure_6 <- cowplot::plot_grid(
  figure_6_body,
  interaction_legend,
  ncol = 1,
  rel_heights = c(1, 0.05)
)

ggsave(
  filename = "outputs/figs/fig6_TE_TT.pdf",
  plot = figure_6,
  width = 7,
  height = 5.6,
  units = "in",
  bg = "white",
  device = cairo_pdf # renders the superscript in "yr-1" correctly
)

```

Figure 6. Temporal change in interaction effects

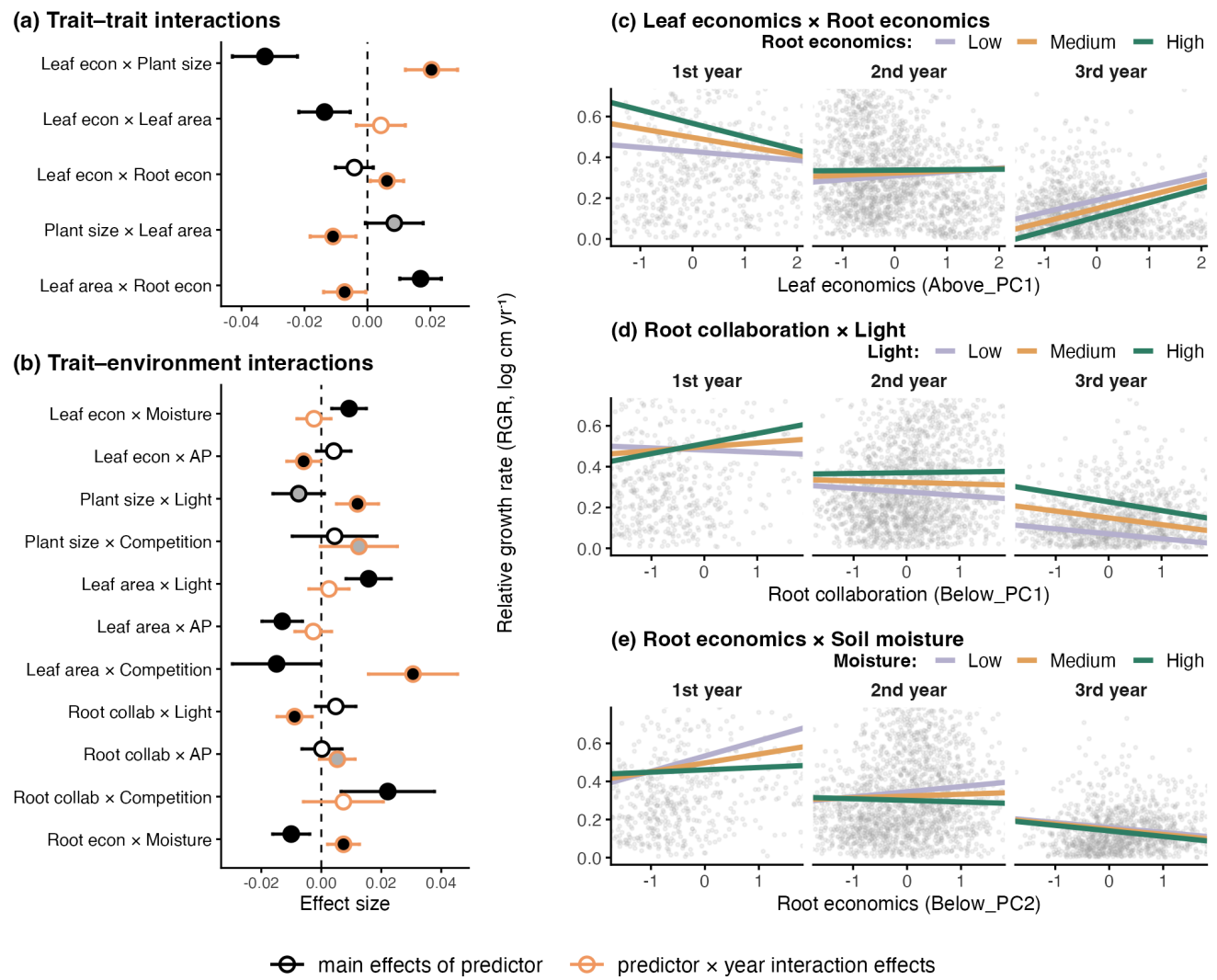
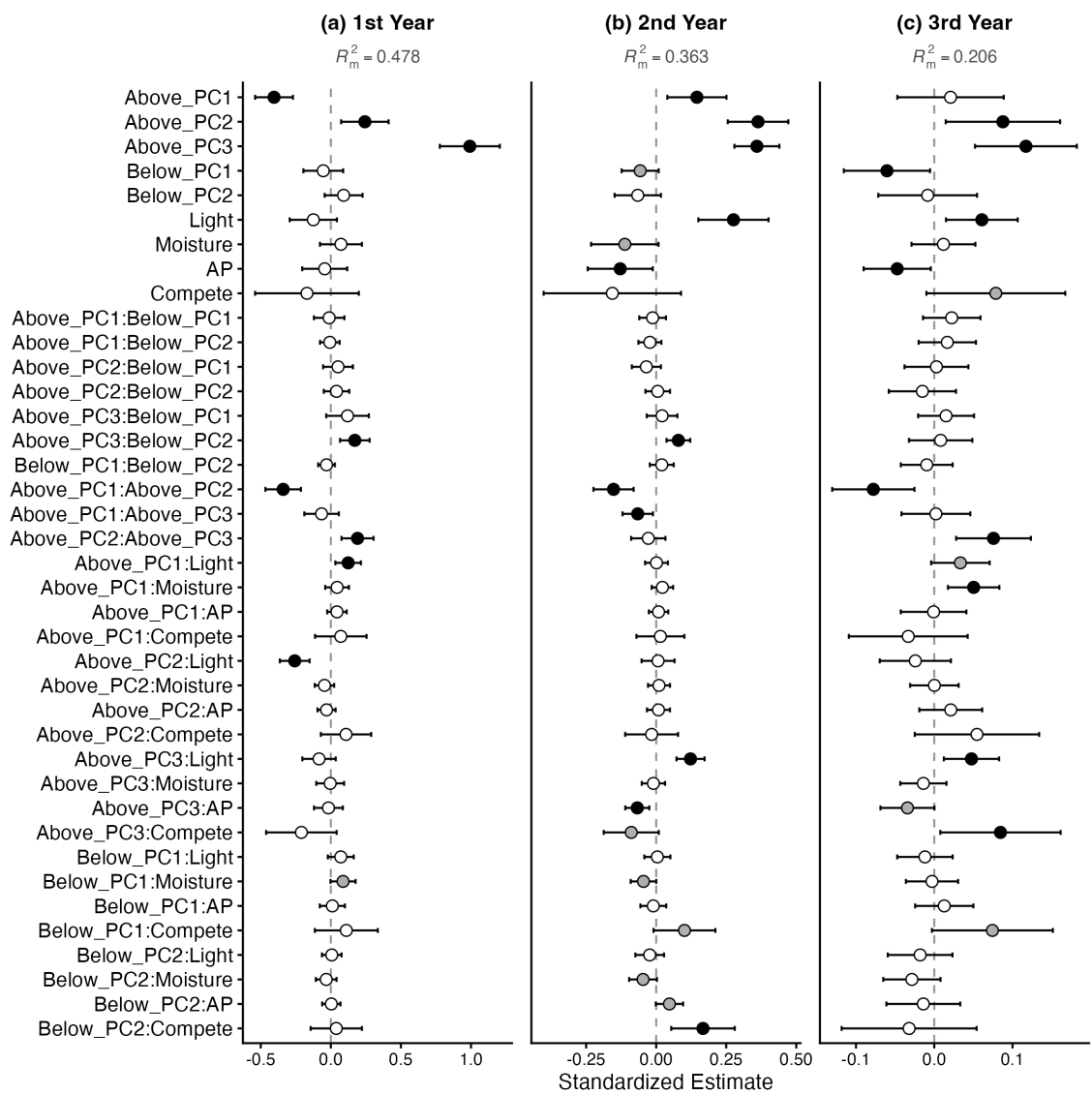


Figure S8. Year-specific global-model robustness

All prespecified trait axes, environmental predictors and interaction terms are fitted simultaneously within each harvest year. This tests whether the patterns from the 24-model framework persist in the corresponding global models.



Reproducibility check

```
expected_outputs <- c(
  file.path(f_dir_name, "fig2_pca_year.pdf"),
  file.path(f_dir_name, "fig3_part_growth_r2.pdf"),
  file.path(f_dir_name, "fig4_sp_var_imp_final.pdf"),
  file.path(f_dir_name, "fig5_trait_effect.pdf"),
  file.path(f_dir_name, "fig6_TE_TT.pdf"),
  file.path(supp_fig_dir, "Figure_S8_year_specific_global_models.pdf")
)
stopifnot(all(file.exists(expected_outputs)))
data.frame(
  Output = basename(expected_outputs),
  Size_kB = round(file.info(expected_outputs)$size / 1024, 1)
)
```

		Output	Size_kB
##			
##	1	fig2_pca_year.pdf	621.1
##	2	fig3_part_growth_r2.pdf	25.2
##	3	fig4_sp_var_imp_final.pdf	18.8
##	4	fig5_trait_effect.pdf	1583.4
##	5	fig6_TE_TT.pdf	405.4
##	6	Figure_S8_year_specific_global_models.pdf	32.3